



DCC: Data-Centric Compilation of Machine Learning Kernels for Processing-In-Memory Architectures

Peiming Yang¹ Sankeerth Durvasula¹ Ivan Fernandez² Mohammad Sadrosadati³
Onur Mutlu³ Gennady Pekhimenko^{1,4} Christina Giannoula⁵

¹University of Toronto & Vector Institute ²Barcelona Supercomputing Center

³ETH Zürich ⁴Nvidia ⁵Max Planck Institute for Software Systems

High-performance Host processors (e.g., GPUs) can integrate Processing-In-Memory (PIM) devices, which can accelerate memory-intensive kernels of Machine Learning (ML) models, including Large Language Models (LLMs), by leveraging the large memory bandwidth available at PIM cores. However, Host processor and PIM cores require different data layouts: Host processor needs consecutive elements distributed across DRAM banks, while PIM cores need consecutive elements within their local banks. This necessitates data rearrangements in ML kernel execution that pose significant performance and programmability challenges, further exacerbated by the need to support diverse PIM devices (e.g., Samsung HBM-PIM, SK Hynix GDDR6-AiM). Current compilation approaches lack systematic optimization for diverse ML kernels and multiple PIM devices, and may largely ignore data rearrangement costs during the compute code optimization step. We demonstrate that data rearrangements and compute code optimization are interdependent, and need to be jointly optimized during the tuning process. To address this, we design DCC, the first data-centric ML compiler for PIM systems that jointly co-optimizes data rearrangements and compute code in a unified tuning process to enable high performance execution. DCC integrates a multi-layer PIM abstraction that enables various data distribution strategies on different PIM backends. DCC enables effective co-optimization of data partitioning strategies with compute loop partitioning schemes. DCC applies PIM-specific code optimizations, and leverages a fast and accurate performance prediction model to select the best-performing code schedule for a given kernel on a target PIM architecture. Our evaluations in various individual ML kernels show that DCC achieves up to 7.68× speedup (2.21× average) on HBM-PIM, and up to 13.17× speedup (3.92× average) on AttAcc PIM, over GPU-only execution. In end-to-end LLM inference, DCC on AttAcc accelerates GPT-3 and LLaMA-2 by 4.52× average (up to 7.71× in LLaMA-2) over GPU. DCC is open-sourced at <https://github.com/SPIN-Research-Group/DCC>.

1. Introduction

Machine Learning (ML) models provide state-of-the-art results across numerous domains including finance [1,2], retail [3], healthcare [4,5], education [6,7], entertainment [8,9], and autonomous systems [10,11]. ML models process increasingly large datasets and comprise both compute-intensive kernels such as General Matrix-Matrix Multiplication (GEMM) and convolutions, as well as memory-intensive kernels such as General Matrix-Vector Multiplication (GEMV) and element-wise operations. For instance, Large Language Models (LLMs) [12–17] contain both compute-intensive fully-connected layers with GEMM kernels and memory-intensive attention layers with GEMV kernels. However, when executed on processor-centric CPU or GPU systems, memory-intensive ML kernels are sig-

nificantly bottlenecked by data movement between off-chip memory and processors [18–23]. Such memory bottlenecks increasingly limit end-to-end ML execution performance.

Processing-In-Memory (PIM) [19,24–41] has emerged as a promising paradigm to alleviate data movement bottlenecks by placing low-power processing units (**PIM cores**) near memory arrays. Numerous works [18–23,28–38,42–49] show that PIM can provide significant performance benefits for memory-intensive ML kernels by reducing data movement costs.

A PIM system includes multiple PIM-enabled memory devices connected to a high-performance *Host* processor (e.g., CPU, GPU, TPU). Near-bank PIM devices tightly couple a PIM core with one or few DRAM banks, exploiting bank-level parallelism to enable large aggregate memory bandwidth. Each PIM core can only access data from its local bank(s). PIM cores of a device may not be able to directly communicate with each other, and inter-core communication can happen via Host. Manufacturers have already started to commercialize near-bank PIM devices (referred to as **PIM backend**). UPMEM PIM [18–20,24,39,40,50] is the first commercialized near-bank PIM device, and can be integrated with CPUs. Samsung HBM-PIM [25] and SK Hynix GDDR6-AiM [26,27] PIM devices have been prototyped and validated, and can be integrated with GPUs. These PIM systems can enable heterogeneous ML execution, where memory-intensive kernels run on PIM cores and compute-intensive kernels run on Host processor.

Host and PIM cores require fundamentally different data layouts to exploit large available memory bandwidth. The Host system distributes consecutive elements *across* multiple DRAM banks to exploit bank-level parallelism, and enables large bandwidth when accessing data at cache line granularity. In contrast, a PIM core can access data only from its local bank(s), and thus in PIM, consecutive elements must be placed *within* the same bank to enable efficient multi-element accesses and maximize local bandwidth. Consequently, the PIM execution of a kernel has three steps: (1) input data rearrangements to place consecutive elements within the same banks for local PIM processing, (2) computation on PIM cores, and (3) output data rearrangements to merge partial results produced from step (2) on PIM cores or prepare output data for Host access by redistributing consecutive elements across banks. These data rearrangements are typically performed via the Host memory bus (outside PIM devices), and thus incur large data movement costs that can dominate end-to-end performance [18–21,51].

Therefore, programming PIM devices is a challenging task [18–21,52]. Programmers must manually craft data rearrangement strategies that balance data movement costs with computation efficiency, which requires deep understanding of both the PIM system and the kernel-specific access patterns of

each ML kernel. Doing so also demands expertise in low-level programming across multiple PIM backends (e.g., [18–21,24–38,42,53]), which may expose different programming interfaces and optimization capabilities. This complexity necessitates compilation tools that automatically and intelligently generate and optimize **data rearrangement strategy** (i.e., how tensor data is partitioned across PIM cores) and **loop-level compute schedule** (i.e., how the computation of nested nested loops is split and assigned to PIM cores) to enhance programmability and minimize end-to-end execution time.

Compilation support with performance optimizations for PIM systems remains in early stages. Existing compilation works for PIM [21,42,52,54–57] lack systematic optimization and auto-tuning for diverse ML kernels and/or support for multiple PIM backends. They largely ignore data rearrangement costs or target only UPMEM PIM. However, UPMEM PIM is designed for CPU systems using DDR4 interfaces, has limited hardware multiplication support and lacks floating-point arithmetic support, making it unsuitable for ML workloads that typically require GPU-PIM co-execution [20,28,33,34,37] and native floating-point operations. ATiM [42] is a search-based tensor compiler that optimizes diverse ML kernels, but supports only UPMEM PIM, and as such is limited to CPU-PIM co-execution. More critically, ATiM enables compute-centric tuning: it optimizes loop-level compute schedule *without* accounting for data rearrangement costs *during* the compute schedule optimization step. However, as we demonstrate in §2.2, compute code transformations and data rearrangements are *interdependent*. Optimizing them in isolation yields sub-optimal performance (See Fig. 2): a compute transformation that appears efficient in isolation may require expensive data rearrangement, while a less efficient compute transformation discarded during compute optimization could enable cheaper data rearrangements. Achieving the best performance requires balancing both costs during tuning rather than optimizing them in isolation.

We propose DCC, a data-centric ML compiler for PIM systems that incorporates data rearrangement strategies into the core of the tuning process, jointly co-optimizing them with loop-level compute schedules to improve end-to-end ML kernel performance and enhance programmability. DCC targets memory-intensive ML kernels suitable for PIM, and has four key components. First, we propose a generic multi-layer abstraction that maps PIM memory hierarchy into a compute hierarchy, where PIM cores form PIM groups. This abstraction decouples the compiler from backend-specific semantics, enabling DCC to support multiple PIM backends and explore diverse data distribution strategies. Second, we design a data-centric schedule generator that constructs all candidate data tensor partitions of an ML kernel across PIM resources and maps them to compute loop partitions. This mapping enables comprehensive co-optimization of data rearrangement strategies with loop-level compute schedules. Third, we integrate a PIM-specific code optimizer that applies data rearrangement and compute code optimizations tailored for PIM systems. This optimizer can be extended with additional PIM-specific optimizations. Fourth, we design a learning-based coupled performance predictor that jointly evaluates data rearrangement and

compute code execution times, and selects the best-performing end-to-end kernel configuration. This predictor provides fast and accurate performance estimates on diverse PIM backends and ML kernels.

We evaluate DCC across diverse memory-intensive ML kernels, tensor sizes, and models using two state-of-the-art PIM backends, HBM-PIM [25] and AttAcc [28]. In individual ML kernels, DCC provides up to 7.68× speedup (2.21× average) on HBM-PIM and up to 13.17× speedup (3.92× average) on AttAcc compared to GPU-only execution. In end-to-end LLM inference on AttAcc, DCC provides up to 7.71× speedup (4.52× average) over GPU and up to 2.74× speedup (1.75× average) over AttAcc’s original implementation.

Overall, we make the following contributions:

- We demonstrate that data rearrangement strategies and loop-level compute schedules must be jointly optimized for memory-intensive ML kernels on PIM systems, and propose DCC, the first data-centric ML compiler that co-optimizes both in a unified tuning process.
- We design a multi-layer abstraction that maps PIM memory hierarchy into a compute hierarchy, and propose a schedule generator that constructs data tensor partitions and maps them to compute loop partitions. We integrate PIM-aware optimizations for data rearrangements and compute code, and employ a learning-based coupled performance predictor to select best-performing end-to-end code schedule for an ML kernel on a target PIM backend.
- We evaluate DCC on two state-of-the-art PIM backends, and show that DCC provides significant performance improvements over best prior works for diverse ML kernels, tensor sizes, and LLMs.
- We open-source DCC in our GitHub repository: <https://github.com/SPIN-Research-Group/DCC>.

2. Background and Motivation

2.1. Processing-In-Memory (PIM) Architectures

Processing-In-Memory (PIM) [19,24–28,33,49,58–60] places low-power processing units near memory arrays, and can alleviate data movement bottlenecks in processor-centric CPU/GPU systems. Near-bank PIM designs tightly couple each core with one (or a few) DRAM banks that can supply data to the core. Near-bank PIM provides larger aggregate memory bandwidth and parallelism compared to near-rank PIM [48,58], where cores are placed at DRAM buffer chip. UPMEM PIM [19,24] is the first commercialized PIM system. Samsung HBM-PIM [25], Alibaba HB-PNM [41] and SK Hynix GDDR6-AiM [26,27] have also been prototyped.

UPMEM PIM is built on DDR4 interfaces for CPU systems, has limited hardware multiplication support and no floating-point arithmetic units [19]. Due to these limitations, UPMEM PIM with CPU-PIM co-execution in ML workloads cannot typically outperform GPU-only execution [19–21,50,51]. In contrast, Samsung HBM-PIM [25], Alibaba HB-PNM [41] and SK Hynix GDDR6-AiM [26,27] are 3D PIM memory devices that can integrate with a high-performance **xPU** processor such as a GPU or TPU, provide hardware multiplication and floating-point units (e.g., FP16), which makes them suitable for ML acceleration. Building on these industry products, numerous research works [28–38] explore enhanced PIM core

microarchitectures and system designs to further accelerate ML kernels. In this work, we target near-bank PIM devices that can be integrated with GPUs/TPUs, have hardware multiplication support and/or floating-point arithmetic units.

We find common characteristics in existing PIM designs [24–41] as shown in Fig. 1. First, a high-performance processor, **Host xPU** (e.g., GPU, TPU) with on-chip cache hierarchy and Host memory typically connects to multiple PIM devices. Second, each PIM device contains multiple processing units (**PIM cores**) that can be organized into **PIM groups**. Each PIM core (e.g., a SIMD unit, GEMV unit) has exclusive access to one or few local memory banks, enabling larger aggregate memory bandwidth and lower latency than Host cores have. Third, PIM cores have register files or scratchpad data memory (referred to as **data memory**). They may support SIMD execution, hardware multiplication, low-precision floating-point or other specialized operations. Fourth, the Host sends PIM instructions that are stored in register files (or instruction cache) of PIM cores. PIM cores execute PIM instructions to move data from/to their local banks to/from registers and/or perform computations (e.g., MUL, ADD) using data stored in registers. Fifth, PIM designs may integrate specialized compute units per PIM group for specific ML kernels. For example, AttAcc [28] provides hardware softmax and accumulator support per PIM group, enabling full attention execution on the PIM side. Finally, PIM cores may not be able to directly communicate with each other, and communication between them typically happens via the Host memory bus.

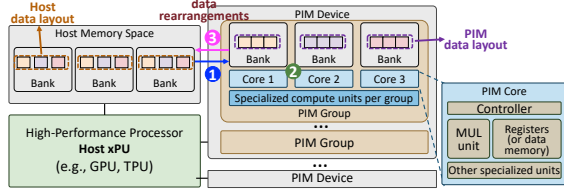


Fig. 1: Near-bank PIM architecture and kernel execution workflow: (1) input rearrangement, (2) computation execution, and (3) output rearrangement steps.

PIM devices typically operate in two modes. Either Host xPU executes a kernel (e.g., compute-intensive kernel) and uses PIM devices as standard DRAM memory for loads/stores. Or, Host xPU offloads a kernel (e.g., memory-intensive kernel) to PIM cores, and PIM cores execute the kernel by accessing their local DRAM banks exclusively. We find that Host xPU and PIM cores require different data layouts to fully leverage their available memory bandwidth, which necessitates software-managed data rearrangements (Fig. 1). Host xPU distributes consecutive elements *across* multiple DRAM banks to exploit bank-level parallelism for large bandwidth, when accessing multiple elements as a cache line. Instead, PIM cores require consecutive elements *within* the same bank: since each core accesses data only from its local bank(s), maximizing local bandwidth requires placing consecutive elements in the same bank to efficiently fetch multiple elements at once as a block. Thus, PIM kernel execution has three steps (Fig. 1): ① input data rearrangements to place consecutive elements within the same bank for local PIM core processing, ② computation execution on PIM cores, and ③ output data rearrangements to either merge partial results from step ② on PIM cores or prepare out-

put data for Host access by redistributing consecutive elements across multiple banks. Data rearrangements are typically performed via the Host memory bus (outside PIM devices), thus incurring significant data movement overheads.

2.2. Need for Data-Centric ML Compiler Support for PIM Architectures

PIM software support remains in early stages, with limited automation and compilation frameworks. SimplePIM [52] optimizes only 1D tensor kernels on UPMEM PIM. PIM-DL [21] effectively supports only the GEMM kernel. CINM [54] integrates multi-level intermediate representations to lower abstractions to PIM, again targeting UPMEM. PIMFlow [55] supports only convolution kernels, automating kernel offloading to GPU or PIM, but without kernel-level tuning optimizations and largely ignoring data rearrangement costs. These works lack systematic optimization and auto-tuning for diverse ML kernels and support for multiple PIM backends. Moreover, most of them target UPMEM PIM, a CPU-integrated DDR4-based device that has no floating-point support and weak multiplication support. This makes UPMEM unsuitable for ML models, where compute-intensive kernels typically run on GPUs/TPUs and memory-intensive kernels on PIM cores may need floating-point arithmetic and strong multiplication units.

ATiM [42] is a search-based tensor compiler designed for UPMEM PIM. ATiM tunes ML kernels on CPU-PIM systems, however, CPU-PIM co-execution on UPMEM systems typically performs worse than GPU-only execution [19,20,39,40,50]. In contrast, GPU-PIM co-execution can deliver substantial performance benefits in ML [28,33,34,37] over GPU-only execution. Although ATiM considers data rearrangement costs, it could not achieve best performance due to its *compute-centric* tuning approach. ATiM has a three-step sequential process: it (i) uses TVM [61] to find and fix a set of templates for compute schedule generation, (ii) generates data rearrangements needed for each template using UPMEM-specific optimizations, and (iii) searches within this *fixed* set of templates to find the best-performing configuration. This sequential approach is suboptimal, because it treats compute schedule optimization and data rearrangement as *independent* problems, while they are fundamentally *interdependent*. A compute schedule that appears efficient in isolation may require expensive data rearrangements, while a slightly less efficient schedule might enable cheaper data rearrangement costs, yielding better end-to-end performance. By fixing compute templates before generating data rearrangements, ATiM explores only a restricted search space and cannot discover configurations where alternative compute schedules paired with different data transformations could minimize total time.

Fig. 2 shows the breakdown of compute (step (2) in Fig. 1) and data rearrangement (steps (1) and (3) in Fig. 1) times for the Reduction and GEMV kernels running on AttAcc PIM [28] (See §5) using various matrix sizes. We evaluate a TVM-based [61] compilation scheme (T) that fairly mirrors ATiM’s approach [42], against a manually-tuned best-performing implementation (B) and our proposal DCC (D). For the TVM-based scheme, we tune TVM via performance profiling of Reduction and GEMV on the PIM devices. TVM selects the best-performing compute schedule templates based on its cost model. For each template, we

generate the optimized data rearrangements and measure end-to-end performance (compute time plus data rearrangement time). We present the configuration with the best performance among TVM’s selected templates. For the manually-tuned best-performing implementation, we manually select some promising data partitioning strategies across PIM cores, and deploy an optimized compute schedule for all of them. Then, we evaluate all of them and present the configuration that provides the minimum total execution time.

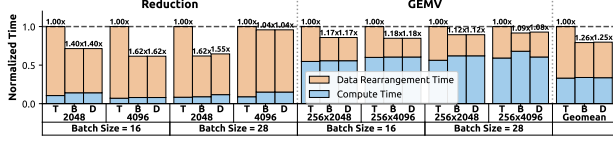


Fig. 2: Normalized breakdown of compute and data rearrangement times in Reduction and GEMV kernels, comparing the TVM-based compilation scheme (T), a manually-tuned best-performing implementation (B) and DCC (D) using various matrix sizes. The numbers on each bar show speedup over T.

We make two observations. First, the TVM-based approach has suboptimal performance. It is on average $1.28\times$ worse than the best manual implementation. This is because the TVM-based approach (followed by ATiM [42]) is compute-centric: it selects compute schedule templates that minimize compute time, while largely ignoring data rearrangement costs. Consequently, data rearrangements are responsible for 64.68% of the total kernel time on average. Second, while the manually-tuned implementation has $1.16\times$ worse compute time than the TVM-based approach, it reduces data rearrangement costs by $1.55\times$ compared to the TVM-based scheme. Thus, it provides better trade-offs that result in better end-to-end performance. Overall, these results indicate that compute schedules and data rearrangements are *interdependent*: obtaining the best performance requires balancing both costs. DCC jointly co-optimizes data rearrangement costs and compute schedules, and provides $1.27\times$ over the compute-centric TVM-based approach, performing very similarly to the best manual implementation.

3. DCC: Overview

DCC (Fig. 3a) is the first data-centric ML compiler for PIM systems that *co-optimizes* data rearrangement strategies together with compute schedule in a unified tuning process. DCC supports diverse ML kernels and multiple PIM backends. Fig. 3a presents a high-level overview of DCC components. Users develop ML kernels using the DCC API (§4.5), which enables execution on target PIM backends. At compile time, DCC analyzes ML kernels in the model and trains its coupled predictor. At run-time, once input tensor dimensions are known (e.g., token sizes in LLMs), DCC uses its pre-trained predictor to select optimized configurations for all PIM-running kernels. The end-to-end kernel configuration orchestrates data loading to the target PIM backend, computation on PIM cores, and final output data with the appropriate layout. DCC has four components.

1. Multi-Layer Abstraction. We design a general multi-layer abstraction for near-bank PIM systems, where PIM cores form PIM groups, which enables DCC to reason about data distribution across memory banks and data processing on PIM groups and cores. Our abstraction (i) enables coarse-grained optimizations at the PIM group level and fine-grained optimizations at the PIM core level, and (ii) decouples the

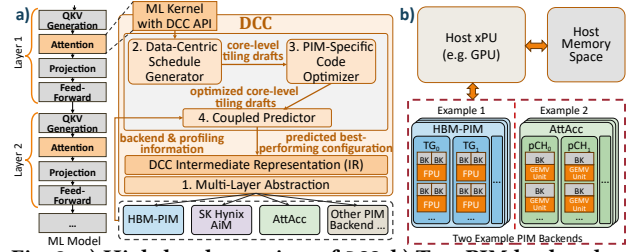


Fig. 3: a) High-level overview of DCC. b) Two PIM backend examples instantiated under DCC’s multi-layer abstraction. compiler from backend-specific semantics, allowing DCC to support multiple current and future PIM backends.

2. Data-Centric Schedule Generator. We follow a *data-first* scheduling strategy that first generates all candidate data partitions across PIM resources, and then maps them to corresponding loop partitions in the compute schedule, creating *tiling drafts*. This approach enables effective *co-optimization* of data rearrangement with loop-level compute schedule.

3. PIM-Specific Code Optimizer. We introduce PIM-specific performance optimizations for data rearrangement strategy and loop-level compute schedule of each tiling draft. This optimizer can be easily extended with additional backend-agnostic optimizations or backend-specific passes to create specialized compiler variants for particular PIM backends.

4. Coupled Performance Predictor. We use a learning-based performance predictor that models the interdependence between data rearrangement costs and compute efficiency and selects the best-performing configuration that minimizes end-to-end execution time. It can provide fast and accurate performance predictions on various PIM backends.

4. DCC: Detailed Design

4.1. Multi-Layer Abstraction for PIM Architectures

To efficiently support multiple PIM backends, we introduce a **general multi-layer abstraction**, which maps the traditional memory hierarchy into a compute hierarchy, allowing DCC to configure where and how each ML tensor is distributed and processed. Our abstractions covers various near-bank PIM backends [19, 24–41], that have the common characteristics described in §2.1. These PIM backends have similar logical organization: they place one PIM core (e.g., specialized acceleration or floating point unit) close to one or a few memory banks. Multiple PIM cores can form **PIM groups** (e.g., cores of the same memory channel or the same DRAM rank can form a group), and the memory bus controller of Host xPU can serve as a controller for all PIM groups. PIM devices may include specialized compute units for each PIM core group (Fig. 1) that can access data from multiple memory banks within the same PIM group. Although the PIM core may have different compute units across different PIM backends, the overall computational model remains consistent across PIM backends. Key differences lie primarily in hardware circuitry design and core microarchitecture rather than execution semantics. Any current or future PIM system that satisfies our proposed multi-layer abstraction can be seamlessly supported by DCC for ML kernel acceleration. Our abstraction has three levels. Two of these levels are within the PIM device.

(1) System Level. Our system-level PIM abstraction consists of three major components (see Fig. 1): the **Host xPU**, **Host**

memory space and the **PIM devices**. The Host xPU (e.g., CPU, GPU, TPU) orchestrates global execution control, data rearrangements, and kernel scheduling. PIM devices perform near-memory computation. The Host memory space is an address space that follows the Host-side data layout, accessible through standard DRAM commands (e.g., LD/ST). It maps to physical addresses in PIM-enabled or normal memory banks. For example (Fig. 3b), in the HBM-PIM system [25], the system level includes the GPU device as Host xPU and multiple HBM-PIM devices. In the AttAcc PIM system [28], it includes the GPU as Host xPU and multiple AttAcc PIM devices.

(2) PIM Group Level. A PIM group represents multiple PIM cores (and their local memory banks) belonging to the same memory channel or rank, depending on the device architecture. A PIM group may include specialized compute units that access data from all banks within the group and/or buffers to temporarily store group-wide data. For instance, in HBM-PIM [25], the group level could include multiple FPU units of one or multiple Thread Groups (TGs) (Fig. 3b) of the same HBM-PIM device. In AttAcc PIM [28], it could include multiple GEMV and accumulator units of one or multiple pseudo-Channels (pCHs) of the same AttAcc PIM device. Each group is managed by the Host xPU memory controller, which may issue group-level instructions. A group-level instruction can be a read/write/compute command (i) that broadcasts to all cores within the group, typically using identical bank address offsets, or (ii) is executed by the specialized compute units of the group. In the latter case, the instruction may access data from shared group-wide buffers or merge partial results produced by PIM cores of the group. Each group has a dedicated memory bus to the Host xPU, enabling DCC to schedule *parallel* data rearrangements across multiple PIM groups. Host can coordinate global data movements across PIM groups or between Host and PIM memory. By scheduling coarse-grained group-level instructions broadcast to all cores within a PIM group, DCC can reduce instruction traffic on the memory controller, and enable parallel operations in the PIM group’s cores.

(3) PIM Core Level. A PIM core represents the processing unit and its local memory bank(s). It can include a SIMD unit or Floating-Point Unit (FPU) or any specialized compute unit. It may have scratchpad memory or register files to temporarily store data for processing. For example, in HBM-PIM [25], the core level includes one single core consisting of an FPU and its register files placed close two (neighboring) memory banks. In AttAcc PIM [28], it includes one GEMV unit placed close to its memory bank. If the Host memory space is mapped to PIM banks, a portion of each bank serves as Host memory, and the PIM backend provides address offsets for the PIM mapping. PIM cores are controlled via bank-level instructions or broadcast group-level instructions sent by the Host memory controller, or instructions stored in dedicated instruction memory per core. By scheduling fine-grained bank-level instructions, DCC enables optimizations on individual PIM cores.

This abstraction enables PIM devices to operate under either group-level or bank-level control for near-bank computation. It also enables DCC to optimize at multiple granularities: coarse-grained at the PIM group level (e.g., effectively partitioning data across groups or scheduling computation on specialized

units of a group) and fine-grained at the PIM core level (e.g., optimizing memory access within a bank or local data layout and computation). It decouples DCC from hardware-specific instruction semantics, thereby enabling DCC to easily support multiple PIM backends that conform to this abstraction.

4.2. Data-Centric Schedule Generation

Traditional ML compilers [61–71] use loop transformations to divide computation into blocks, and improve data locality. DCC adopts a **data-first schedule** strategy that inverts this process: it 1) generates all candidate data partitions of data tensors (called **data tiles**) across PIM groups and cores (following its multi-layer abstraction), 2) maps data tiles to their corresponding loop partitions (ranges of loop variables) in the compute code (called **compute tiles**), 3) optimizes each configuration’s performance, 4) then uses its predictor to evaluate data-compute tile mappings, 5) selects the best-performing schedule, and 6) generates PIM instructions for the best-performing end-to-end kernel configuration for the target PIM backend.

To find the best data-compute schedule for a given kernel and hardware configuration, DCC generates a comprehensive set of **tiling drafts**, each representing a potential mapping between tensor sizes, computation loops, and PIM hardware resources. The generation process includes four stages shown in Fig. 4 for an example ML kernel and explained next.

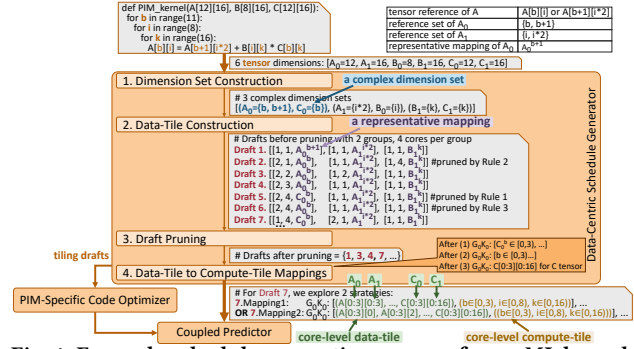


Fig. 4: Example schedule generation process for an ML kernel.

1. Dimension Set Construction. To jointly co-optimize compute-data, we need to correlate the data tensors with the loop-level compute schedule. For a k -dimensional tensor A , we map its dimensions $A_0, A_1, \dots, A_k$ to the loop-level compute schedule: we leverage loop variables as intermediaries to associate tensor dimensions to mapping functions. Specifically, for a tensor A with tensor reference $A[b+1][i*2]$, where A_0 is the first dimension and A_1 is the second dimension, we use the variables and constants to build mapping functions for each dimension: e.g., the mapping functions $F(b)=b+1$ and $F(i)=i*2$ are for the dimensions A_0 and A_1 , respectively. A tensor may have multiple tensor references in nested loops, e.g., tensor A in Fig. 4 has two references $A[b][i]$ and $A[b+1][i*2]$, creating multiple mapping functions for the same tensor dimension. The dimension A_0 is associated with both $F(b)=b$ and $F(b)=b+1$. We build all mapping functions for each tensor dimension and collect them into a **reference set**. The reference set for dimension A_0 is $\{b, b+1\}$, and for A_1 is $\{i, i*2\}$. The reference set for the tensor dimension C_0 is $\{b\}$, and for C_1 is $\{k\}$. Next, when two or more tensor dimensions share the *same* loop variable in their reference sets, they are grouped

into a **complex dimension set**. For example, dimensions A_0 and C_0 both contain variable b in their reference sets, thus we group them as ($A_0=\{b, b+1\}$, $C_0=\{b\}$) to be processed jointly, since these dimensions must share the same loop indexing values, i.e., b takes the same indexing values in loop iterations. Tensor dimensions that do not share variables with any other dimension form simpler dimension sets.

2. Data-Tile Construction. We explore all candidate data tile options through exhaustive depth-first search with memoization, constructing data tiles for each dimension set. For the i -th dimension set among n dimension sets, given k available PIM groups and m available cores per group, we try all possible PIM allocations, i.e., $[1, k]$ PIM groups and $[1, m]$ cores per group. The available PIM groups and cores for the $(i+1)$ -th dimension set are computed recursively as $avail_resources_{i+1} = \frac{avail_resources_i}{allocated_resources_i}$. Memoization caches all valid allocation solutions for each subproblem, enabling reuse when the same PIM resource configuration is encountered again during search. Once the allocation of PIM resources is done for all dimension sets, we create multiple tiling drafts as follows. Each draft has n parts, one for each dimension set. Each part includes three parameters: [the number of groups, the number of cores, a representative mapping]. The **representative mapping** is an assignment of a mapping function (e.g., $b+1$) to a tensor dimension (e.g., A_0). We create drafts via exhaustive search for all possible assignments of representative mappings for a given dimension set. For instance, in Fig. 4, the first draft has three parts for its three dimension sets. The first part is $[1, 1, A_0^{b+1}]$, which represents 1 PIM group, 1 core per group, and the representative mapping A_0^{b+1} . The second part $[1, 1, A_1^{i*2}]$ represents 1 group, 1 core per group, and the representative mapping A_1^{i*2} . Similarly, the third part is $[1, 1, B_1^k]$. When this step finishes, we have generated all possible tiling drafts for all possible data tiles.

3. Draft Pruning. DCC applies three pruning rules to eliminate unpromising tiling drafts and reduce tuning time, while maintaining high performance. **Rule 1.** We remove redundant tiling drafts that have the same data tiles and equivalent representative mappings for all dimension sets. In Fig. 4, the 5th draft $[[2, 4, C_0^b] \dots]$ and the 6th draft $[[2, 4, A_0^b] \dots]$ are redundant. In their first part (which relates to the first dimension set), they have the same data partition, i.e., 2 PIM groups and 4 cores per group, and equivalent representative mappings, i.e., their representative mappings C_0^b and A_0^b use the same mapping function $F(b)=b$, and thus they generate *identical* data partitions. The second and third parts of these tiling drafts are identical. Thus, one of these two drafts, i.e., the 5th draft, is removed. **Rule 2.** PIM backends [25–28, 32–35] support SIMD-style instructions that enable PIM cores to fully leverage the large memory bandwidth available in their local bank(s). For example, HBM-PIM [25] supports 16-way SIMD instructions. In a PIM backend with d -way SIMD instruction support, the draft pruner speculatively eliminates drafts where the size of the dimension set assigned to a PIM core is not a multiple of d : these drafts cannot fully utilize the core’s local bandwidth and hardware datapaths designed for d -way execution. In Fig. 4, let us assume a PIM backend with 16-way MUL instructions, and consider the dimension set

($B_1=\{k\}$, $C_1=\{k\}$) for the computation $B[i][k] \times C[b][k]$. The 2nd draft assigns 4 cores to the dimensions B_1 and C_1 (the first dimensions of tensors B and C), both of size 16, so each core must process 4 elements along the dimension assigned to it. Since 4 is not a multiple of the 16 (16-way MUL instruction), this draft is pruned. If all drafts fail to satisfy the d -way instruction alignment requirement, or if the PIM backend does not support SIMD instructions, DCC skips this pruning rule. **Rule 3.** Within a PIM group, execution performance is determined by the worst-performing core in the group. When comparing multiple drafts, if the worst per-core performance is the *same* across all corresponding PIM groups for each part of the draft, those drafts are execution-performance equivalent. In this case, only one draft needs to be kept, while the remaining drafts are pruned. To apply this pruning rule, we estimate per-PIM-core performance using the dimension partition size assigned to each core. This partition size is derived from the dimension included in the representative mapping of each part of the draft. The PIM core with the largest dimension partition size within a PIM group is taken as the worst-case performance representative for that group. In Fig. 4, the 4th and 6th drafts each have three parts. For the first part, whose representative mapping is A_0^b , both drafts have 2 PIM groups. The worst per-PIM-core performance for the A_0 dimension derived from A_0^b for the corresponding PIM groups in both drafts is the *same*: in the 4th draft, it is $\text{round}(\frac{A_0}{groups \times cores}) = \text{round}(\frac{12}{2 \times 3}) = 2$, and in the 6th draft, it is $\text{round}(\frac{A_0}{groups \times cores}) = \text{round}(\frac{12}{2 \times 4}) = 2$. The remaining two parts of the 4th and 6th drafts yield the same worst per-core performance across all their PIM groups. Since the two drafts are execution-performance equivalent, one is arbitrarily pruned, i.e., the 6th draft is pruned.

4. Data-Tile to Compute-Tile Mappings. Once the drafts are pruned, DCC constructs core-level **data-tiles** and maps them to core-level **compute-tiles**. The remaining drafts include only the high-level tiling of each part, i.e., the number of PIM groups, PIM cores per group, and the representative mapping of the corresponding dimension set. However, we need to (1) lower the high-level tiling to core-level data-tiles, (2) map the core-level data-tiles to core-level compute-tiles, and (3) determine the necessary data indices of all tensors corresponding to loop variable values of the compute-tiles.

(1) For each part of a draft (which corresponds to a dimension set), we lower the high-level tiling to core-level data-tiles. Given the number of PIM groups, PIM cores per group, and the representative mapping of a dimension set, the data-tile for core K_j of the PIM group G_i is computed by dividing the size of the dimension (the dimension is derived from the representative mapping) by the product of the number of PIM groups and the number of PIM cores per group, i.e., $\frac{dimension_size}{groups \times cores}$. In Fig. 4, the first part of the 7th draft $[1, 4, C_0^b]$ has 1 PIM group and 4 PIM cores per group, and the representative mapping C_0^b (which corresponds to the dimension set ($A_0=\{b, b+1\}$, $C_0=\{b\}$)). The data-tile size for each core along the dimension C_0 is $\frac{C_0}{groups \times cores} = \frac{12}{1 \times 4} = 3$. Thus, the data-tile for the PIM core K_0 of the PIM group G_0 for the first dimension set is $C_0 \in [0, 3)$ (Fig. 4). We similarly compute the data-tiles for all parts of the draft (corresponding to all dimension sets) and for all cores.

(2) For each part of the draft (which corresponds to a dimension

set), we map the core-level data-tile to a core-level compute-tile by determining the loop range of the associated variable for the kernel loops. Given the data-tile of core K_j of PIM group G_i and the mapping function (e.g., $F(b)=b$) derived from the representative mapping, the compute-tile is the set of all values of the associated variable (e.g., b) for which the mapping function result falls within the data-tile (e.g., $F(b) \in [0,3)$). In Fig. 4, for the first part of the 7th draft, core K_0 of the PIM group G_0 has data-tile $[0,3)$ (from step (1)) for the first dimension set, and the mapping function is $F(b)=b$. The compute-tile is thus the set of values of b satisfying $F(b) \in [0,3)$, which gives $b \in [0,3)$. Similarly, we calculate all loop ranges for all loop variables across all cores.

(3) Once the compute-tile for each core is determined, DCC derives the **tensor data indices** (e.g., $A[0:3][0:3]$) that each core must access for **all** the tensor dimensions, corresponding to the loop variable values of the compute-tile. In Fig. 4, for the 7th draft, DCC derives tensor data indices for all tensor dimensions A_0, A_1, B_0, B_1, C_0 and C_1 . When the required indices of a tensor are non-contiguous along a dimension, DCC explores two alternative strategies. Strategy (i) expands the indices of this dimension to cover a contiguous range, adding the missing or straddling values among the required indices (e.g., in Fig. 4 in *7.Mapping1* the indices of A_1 are expanded from $\{0, 2\}$ to $[0:3]$). Strategy (ii) keeps only the required indices as-is (e.g., in *7.Mapping2* the index set of A_1 is $[0]$ and $[2]$ (kept as it is)). Since each strategy has different performance implications, DCC generates both and selects the better-performing one.

4.3. PIM-Specific Code Optimizer

Once tiling drafts are generated, data is mapped across PIM banks and compute loop partitions is mapped to PIM cores. DCC then applies a **PIM-specific code optimizer** for *each* tiling draft that further optimizes performance. We exploit common PIM architectural characteristics to enable one optimization that reduces data rearrangement costs and one that improves the execution efficiency of loop-level compute schedule.

I) Data Rearrangement Optimization. Host and PIM cores require different data layouts to achieve high performance. In Host memory, contiguous data needs to be distributed across multiple memory channels to exploit channel-level parallelism. For example, on a two-channel CPU, a 1KB block is divided into 16 cache blocks (indexed 0-15), with odd-indexed lines mapped to one channel and even-indexed lines to the other channel. This enables *parallel* sequential reads across multiple memory channels. In contrast, PIM cores require contiguous data to be stored in a single memory bank, i.e., using one single memory channel for writes, since each PIM core can only process data stored in its local bank(s). This creates the following data rearrangement challenge: (i) reading contiguous data sequentially from Host memory exploits Host channel parallelism, but results in writes that use only one PIM memory channel, underutilizing PIM bandwidth, while instead (ii) using multiple PIM channels for parallel writes requires simultaneous reads from *non-contiguous* Host addresses, potentially causing Host channel conflicts and degrading read performance. To enable channel parallelism for both Host reads and PIM writes, DCC exploits the controllable on-chip memory (e.g., GPU shared memory) to reorganize data within the Host. For a PIM device

with N channels, the compiler calculates the data block size as $B = \frac{\text{on-chip memory size}}{N}$. DCC then serially reads N blocks of size B from Host memory to on-chip memory, followed by parallel write operations that distribute these blocks across all N PIM channels simultaneously. This approach transforms data movement into N sequential read operations from Host channels and N parallel writes to PIM channels, leveraging channel parallelism in both reads and writes. If there is no controllable on-chip memory in the system (e.g., on CPUs), DCC performs data rearrangement using the PIM backend's default data copy or DMA interfaces. For PIM backends with specialized layout constraints (e.g., alignment or interleaving requirements), DCC applies additional compilation passes for performing required data layout transformations.

II) Compute Code Optimization. PIM backends [25–28] can use specialized DRAM commands to control computation on PIM cores. However, when all PIM cores perform computations in parallel, the Host memory controller must issue significantly more commands than in conventional DRAM, creating a control bottleneck due to the limited memory bus bandwidth. To address this, PIM backends [25–28] introduce group-level commands that provide SIMD-style control: instead of issuing individual compute commands to each PIM core, a single compute command is broadcast to *all* cores within a *group*, with each core operating on its own local data, significantly reducing command bus overhead. DCC employs a hierarchical command generation strategy leveraging both **group-level** and **bank-level** control to balance efficiency and flexibility. DCC detects and prioritizes **group-level commands** wherever possible. Since DRAM command formats constrain group-level commands to use identical address offsets for all cores in the group, DCC pads each core's tensor data to the same size to ensure consistent local addressing. This way DCC allows the memory controller to issue one command per PIM group, significantly reducing command traffic. If the backend lacks group-level command support or when cores need to access different address offsets, DCC generates *parallel bank-level commands*, achieving fine-grained, bank-level parallelism. This hierarchical command generation strategy adapts to different PIM backends, and minimizes command traffic.

Our optimizer applies PIM-specific optimizations that accelerate performance while remaining backend-agnostic. It has a modular design that facilitates the integration of additional optimizations shared across multiple PIM backends. Developers can also extend DCC with custom optimization passes tailored to specific PIM backends, creating specialized compiler variants (e.g., DCC +HBM for HBM-PIM-specific optimizations).

4.4. Coupled Performance Predictor

Although pruning substantially reduces the number candidate drafts, the schedule generator can still produce thousands of valid tiling drafts. Profiling all drafts on the target PIM system can be prohibitively expensive. To efficiently identify the best draft, DCC employs a **coupled learning-based performance predictor** that estimates end-to-end execution time. The coupled predictor serves two purposes: (i) co-estimating data rearrangement and compute costs to find the best tiling draft, and (ii) providing fast and accurate predictions across multiple PIM backends. We choose a learning-based model (already effec-

tively adopted by widely-used ML compilers [61,72,73]) that can be easily trained (retrained or fine-tuned) for a new PIM backend, and provide both low latency and high prediction accuracy. We use an XGBoost model [74] to predict end-to-end kernel time, because XGBoost has been proven highly effective for compiler cost modeling in prior works [61,72,73,75–78].

While analytical models can achieve sufficient accuracy with PIM device-specific formulas and parameters, they are difficult to generalize across multiple PIM backends with competitive accuracy and maintain as PIM architectures evolve. Moreover, unlike neural predictors (e.g., these used in [72]), XGBoost is lightweight in memory and compute requirements.

The coupled predictor supports both static and dynamic tensor sizes. For static sizes, where model and input dimensions are fixed during inference, DCC trains the predictor offline and configures the best-performing schedule during model initialization (similarly to prior ML compilers [61,72,73]). For dynamic inputs, which occur in LLMs, DCC finds and selects the best-performing schedules at inference time.

Offline Training: Given model-defined or user-provided tensor sizes, DCC samples a diverse subset of drafts across different tensor sizes from the schedule generation step. Each draft is profiled on the target PIM backend to measure execution time as labels, with draft’s configurations and backend information as inputs to the XGBoost model. After training, DCC predicts performance for all drafts at given tensor sizes of ML kernels and records the best-performing draft in a lookup table. When multiple drafts have identical predicted performance, one is selected randomly. This training occurs *once* per PIM backend using given ML models and tensor sizes. Offline training for all ML kernels across all tensor sizes, ML models, and PIM backends used in our evaluation takes only ~ 67 seconds. This cost is negligible and is amortized across multiple users’ inference requests for that ML model. At runtime, DCC *directly* uses the best-performing draft for recorded tensor sizes.

Dynamic Prediction: When a new tensor size appears during inference, DCC generates the corresponding tiling drafts and uses the predictor to estimate performance, recording the best-performing schedule in the lookup table for future use. In ML models with dynamic tensor sizes, e.g., evaluation of LLMs in Fig. 10, the cost to generate drafts on-the-fly and estimate performance is accounted for in total time. Optionally, DCC can support training the predictor at inference time: it can profile a small batch of new drafts, and use the (updated) predictor to estimate performance for remaining drafts.

4.5. Programming Interface & Intermediate Representation (IR)

DCC provides a Python [79] interface with PyTorch [80], described in Table 1, to deploy, integrate and compile ML kernels for PIM execution. DCC compiles Python kernels marked with the `@DCC_kernel` annotator. The programmer writes a Python kernel that describes tensor computations, specifying tensor shapes and data types needed at compile time. All parameters and return values must be PyTorch tensors or scalar constants. The kernel body describes computations over tensors using affine loop nests, elementary arithmetic operators, and backend-supported instructions, including operations on tensors of varying numeric precision (e.g., integer and floating-

point) and conditional operations. If there are multiple loop nests that share the same mapping function and loop range, DCC merges them using the same complex dimension set.

Interface	Description
<code>@DCC_kernel</code>	Annotator to define PIM-running kernels.
<code>DCC.Tensor</code>	Tensor class used in kernel. It can be converted to <code>torch.Tensor</code> .
<code>DCC.Layer</code>	Class to define a new PIM-running layer.
<code>DCC.Kernel</code>	Class to represent a PIM-running kernel.
<code>DCC.initKernel()</code>	Function to initialize the kernel for given input sizes. It returns a <code>DCC.Kernel</code> object.
<code>DCC.Kernel.update()</code>	Function called when the best tiling draft has been determined.
<code>DCC.Kernel.preLoad()</code>	Function to load data to PIM devices and return a PIM address. It also supports partial updates of tensors in PIM memory with address offset.
<code>DCC.Kernel()</code>	Function to execute a PIM-running kernel with <code>torch.Tensor</code> or PIM address.
<code>DCC.setModel()</code>	Function to register the model and extract model-level metadata necessary for tuning.

Table 1: The DCC Python Programming Interface.

Users can integrate a kernel executed on PIM into an ML model using `DCC.Layer` to create ML layers that replace existing model layers. PIM-running kernels must be initialized with input tensor sizes. DCC provides `DCC.Tensor` and `DCC.Tensor.zero()` to create and initialize temporary tensors. After adding PIM-accelerated layers to the model, DCC initializes necessary model metadata, infers all tensor sizes, and performs offline training. At inference, when a request is received, DCC asynchronously infers tensor sizes for all kernels executed on PIM, and selects their best-performing schedules. Users can override the function `DCC.Layer.update()` to pre-load data to PIM devices using `DCC.Kernel.preLoad()` before running the kernel. When `forward()` is called, the kernel is executed on PIM cores with the input data and returns the results as a `torch.Tensor`. DCC synergistically works with xPU compilers (e.g., ML compilers [61,63] for GPUs) via PyTorch. During the optimization for xPU, the PIM layer will be configured as non-fusible to enable intra- and inter-kernel optimizations. DCC compiles and optimizes this layer for the PIM backend.

Fig. 5 shows an example of replacing the QKV generation layer in GPT-3 13B model with a PIM-accelerated layer. Lines 1–8 define the kernel (running on PIM) with for loops. Line 10 creates class `QKV_Layer` inheriting from the `torch.nn.Module` and `DCC.Layer`. In lines 11–14, DCC initializes the kernel and model parameters. Lines 16–18 pre-load weights and bias to PIM devices after having selected the best-performing schedule. Line 21 executes the kernel with the given input and pre-loaded data. Lines 23–28 replace the original QKV layer in the loaded GPT-3 model and handle inference requests.

DCC parses Python kernels marked `@DCC_kernel`, compiles them to its *data-centric* Intermediate Representation (IR), and then generates low-level backend instructions based on the hardware intrinsics supported by the target PIM backend. If the target PIM backend does not support a specific operation, DCC reports a compilation error. The DCC’s IR constructs are described in Table 2.

IR Construct	Description
<code>@hardware_platform[G,C]</code>	Defines the PIM hardware configuration for the annotated DCC kernel function, i.e., the number of PIM groups and PIM cores per PIM group.
<code>func @name(%args:<shape>dtype):<shape>dtype</code>	Defines a PIM kernel function with typed input/output.
<code>complex_dimset %ds [%dim[0]...]</code>	Constructs a complex dimension set as %ds with one or more mapping functions %dim[%idx].
<code>%dim[%idx] = %t.dim[%axis] ->expr</code>	Declares a mapping function %dim[%idx] on a tensor dimension %t.dim[%axis] as <i>expr</i> .
<code>[representative]</code>	Marks a mapping function in a complex dimension set as the representative mapping.
<code>%ds.range = (start, end, step)</code>	Specifies the iteration range of the loop variable in a complex dimension set %ds.
<code>data_tile %dt: (g,c) -> (g,c,t)</code>	Constructs a data tile %dt as PIM groups (g), PIM cores (c), and the generated tile size (t). g and c are included in the draft description.
<code>compute_tile %ct: (g,c,t) -> [[R₀...]]</code>	Generates a compute tile %ct based on the data tile %dt. Each %ct is a 2D array containing the loop variable ranges R _i of all i partitions of a tensor dimension.
<code>tensor_tile %tt: %ct -> [[T₀...]]</code>	Generates all necessary data indices of all tensors from a compute tile. Each %tt is a 2D array describing the necessary indices T _i of all i partitions of a tensor dimension.
<code>%t_core = rearrange %t -> %tt</code>	Rearranges tensor data between Host data layout and PIM data layout in local banks. %t_core contains the core-level local data reference of the tensor %t.
<code>[to_PIM_core/to_host]</code>	Shows the direction of data movement.
<code>parallel_PIM (groups, cores) [...]</code>	Launches parallel execution across all PIM groups and PIM cores concurrently.
<code>loop (x₁...x_n) in (%ct₁...%ct_n)</code>	Iterates over the compute-tiles %ct ₁ ...%ct _n using n loop variables x ₁ ...x _n to drive PIM core-local computation.

Table 2: IR constructs of DCC and their descriptions.


```

1 @DCC_kernel
2 def PIM_kernel(weight, bias, x): # define PIM kernel
3     y = DCC.Tensor.zero([x.size(0), weight.size(0)])
4     for b in range(y.size(0)):
5         for i in range(y.size(1)):
6             for j in range(x.size(1)):
7                 y[b][i] += x[b][j] * weight[i][j] + bias[j]
8     return y
9
10 class QKV_Layer(torch.nn.module, DCC.Layer):
11     def __init__(self, layer, input_sizes): # init kernel and weight
12         self.kernel = DCC.initKernel(PIM_kernel, input_sizes)
13         self.weight = layer.weight
14         self.bias = layer.bias
15
16     def update(self, best_draft): # pre-load data
17         self.PIM_W = self.kernel.preLoad(best_draft, self.weight)
18         self.PIM_b = self.kernel.preLoad(best_draft, self.bias)
19
20     def forward(self, x): # execute PIM kernel
21         return self.kernel(self.PIM_W, self.PIM_b, x)
22
23 model = torch.load("GPT3-13B.model") # load model
24 in_sizes=[model.qkv_0.weight.size(), model.qkv_0.bias.size(),
25           [1, model.hidden_size]]
26 model.qkv_0 = QKV_Layer(model.qkv_0, in_sizes) # replace QKV layer
27 model = DCC.setModel(model) # collect model information
28 output = model(get_request()) # run the inference

```

Fig. 5: An example of adding DCC kernels to a GPT3 model.

Fig. 6 illustrates how DCC compiles a kernel (Fig. 6a) from the Python code to its IR of data- and compute-tiles for a given schedule, and subsequently to final backend instructions. For a given schedule produced by the schedule generator (§4.2), the IR generator produces an IR program (Fig. 6b). Lines 1–2 declare the available PIM hardware resources and the typed input and output tensors. Lines 4–13 create the *complex dimension sets* with designated *representative mappings* (§4.2.1), and declare their iteration ranges (lines 8, 13). Lines 15–16 generate the *data-tiles* by binding the dimension sets to PIM hardware resources (groups, cores) (§4.2.2) and calculating the per-core data-tile sizes (§4.2.4 step (1)). Lines 17–18 derive the *compute-tiles* from the *data-tiles*, and generate the iteration range for each loop variable at the core level (§4.2.4 step (2)). Lines 19–20 derive the necessary *tensor data indices* at the core level for all tensors, corresponding to loop variable values (§4.2.4 step (3)), and implementing the (ii) tensor indexing strategy in this example. Lines 22–29 derive the end-to-end execution flow (See Fig. 1): performing input data rearrangements for the input tensors (lines 22–24), executing the kernel computation concurrently (parallel) across all PIM cores in tiled compute domains (lines 25–28), and performing output data rearrangements for the output tensor results (line 29). Once IR generation is complete, DCC parses the IR program and translates it to the target backend instructions: Fig. 6c and Fig. 6d show the generated backend instructions of the tensor computation of line 27 of Fig. 6b for the AttAcc and HBM-PIM backends, respectively.

5. Evaluation

Simulation Methodology. We modify and use the open-source AttAcc simulator [28] with Ramulator 2.0 [81,82] (based on [83,84]) We evaluate data movement costs to/from PIM devices using DRAM commands (i.e., LD/ST) simulated in Ramulator 2.0. We modified the simulator to support two PIM backends, i.e., AttAcc [28] and HBM-PIM [25], with its corresponding DRAM compute commands [25]. We simulate a heterogeneous platform with a NVIDIA A100 GPU and 5 PIM-enabled HBM-type devices, corresponding to 80GB GPU mem-

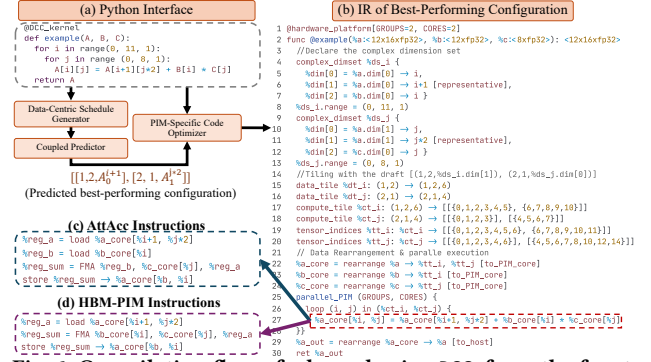


Fig. 6: Compilation flow of a kernel using DCC: from the front-end code to its IR of data- and compute-tiles, and then to back-end instructions.

ory. Our platform configuration is consistent with prior state-of-the-art works [28,33]. We evaluate HBM3 [85,86] memory with 5.2Gbps per pin and running at 333MHz. Each HBM has 16 pseudo-channels (pCH), 2 ranks per pCH, 2 banks groups per rank and 4 banks per bank group (64 banks per pCH). We follow the timing parameters that AttAcc used: $t_{CK}=0.79$, $t_{RCD}=19$, $t_{RP}=19$, $t_{CL}=19$, $t_{CCD}=4$, $BL=2$. In AttAcc [28], each bank of each HBM-type device is equipped with a GEMV unit and each channel has a softmax unit. In HBM-PIM [25], every two banks of each HBM-type device share a 16-way FP16 FPU and two 16×256 -bit GRF registers (one per bank).

ML Kernels and Models. We evaluate seven memory-intensive ML kernels. In AttAcc, we evaluate the general-matrix-vector-multiplication (GEMV), reduction (RED) and attention (ATTN) kernels. In HBM-PIM, we evaluate the GEMV, RED, vector addition (VA) and RELU kernels. Attention requires a softmax unit which only exists on AttAcc. However, AttAcc does not have near-bank compute units that could support and run RELU and VA. For GEMV and ATTN, we use input size 128, the most common per-head dimension in LLMs. We also evaluate end-to-end inference using GPT3-13B and LLAMA2-33B models. We evaluate FP16 data type.

Comparison Points. We evaluate five comparison points. (1) **GPU**: all kernels run on an A100 GPU configured in AttAcc and Ramulator 2.0 simulators. (2) **AttAcc**: AttAcc’s [28] default open-source implementation that distributes different batches or attention heads across 16 pCHs, partitions the first tensor dimension across 16 bank groups per pCH, and partitions the second tensor dimension across 4 banks per bank group. (3) **AttAcc+DCC**: we enable DCC compiler on AttAcc. (4) **HBM-PIM**: we use AttAcc’s data distribution. (5) **HBM-PIM+DCC**: we enable DCC compiler on HBM-PIM.

For offline training, we configure XBoost with `learn_rate=0.1`, `max_depth=8`, `num_boost_round=5000`. For all workloads, we use 15% of the pruned drafts as training samples, and 85% as test set. The compilation time (offline draft generation, training, prediction) for all evaluated workloads is ~ 67 seconds in total using a system with 32-core AMD EPYC 7513 CPU, 128GB DDR4 memory and NVIDIA A100 GPU.

5.1. ML Kernel Performance

Fig. 7 shows the speedup of AttAcc and AttAcc+DCC over the GPU baseline in various ML kernels, when varying the batch size, number of heads and tensor sizes.

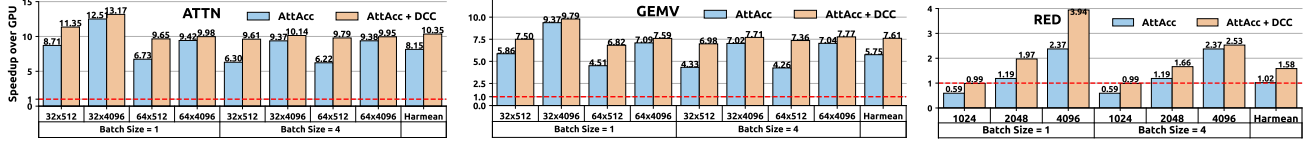


Fig. 7: Speedup of AttAcc and AttAcc+DCC over GPU for ATTN, GEMV and RED kernels, when varying the tensor sizes.

We make three key observations. First, AttAcc significantly outperforms GPU across most kernels, achieving $10.35\times$ for ATTN, $7.61\times$ for GEMV, and $1.58\times$ for RED on average, with the exception of RED at tensor size 1024 where GPU performs better. AttAcc leverages high aggregate PIM bandwidth and integrates specialized units (GEMV, softmax, and accumulator) that provide hardware-level support for these operations. Second, DCC provides further performance improvements over AttAcc: $1.23\times$, $1.26\times$, and $1.48\times$ average speedup, and up to $1.57\times$, $1.73\times$, and $1.66\times$ peak speedup for ATTN, GEMV, and RED kernels, respectively. For RED with tensor size 1024, AttAcc underperforms GPU by $0.59\times$, while with DCC provides almost the same performance as GPU. Third, with DCC, performance scales well in RED as tensor size increases. In RED, data rearrangement costs dominate the total time (see Fig. 12 in §5.4) and with larger tensor sizes, DCC explores a larger search space, allowing it to more effectively optimize data rearrangement costs. In RED, with batch size of 1 and tensor size of 4096, DCC provides a large speedup of $3.94\times$. Overall, DCC significantly accelerates end-to-end time on the state-of-the-art AttAcc backend by up to $13.17\times$ ($3.92\times$ on average) over GPU across diverse ML kernels, batch sizes and tensor sizes, thanks to its comprehensive data-compute co-optimization.

Fig. 8 shows the speedup of HBM-PIM and HBM-PIM+DCC over GPU in various ML kernels, when varying the batch size and tensor sizes. We make four key observations. First, DCC significantly improves HBM-PIM performance by $1.53\times$, $2.09\times$, $1.64\times$, and $1.54\times$ for GEMV, RED, VA, and RELU on average, respectively, enabling HBM-PIM to further outperform GPU by $5.59\times$, $1.35\times$, $2.07\times$, and $2.66\times$, respectively. Second, in large batch sizes and tensor sizes, AttAcc’s data distribution that is used on both AttAcc and HBM-PIM backends is highly optimized over GPUs. Thus, in both HBM-PIM and AttAcc backends (Figs. 7, 8), DCC exploits limited optimization opportunities, and enables smaller speedups over GPUs. Third, when using the same batch and tensor size configurations, DCC provides $1.53\times$ and $2.09\times$ average speedup over HBM-PIM in GEMV and RED, respectively, and $1.26\times$ and $1.48\times$ average speedup over AttAcc in GEMV and RED, respectively. DCC provides greater performance improvements on HBM-PIM than on AttAcc, because HBM-PIM is less optimized in hardware than AttAcc for GEMV, i.e., HBM-PIM includes general SIMD units, while AttAcc has specialized GEMV units. Fourth, for RED with tensor sizes 1024 and 2048, HBM-PIM underperforms the GPU by $0.48\times$ averaged across all batch sizes. This is because HBM-PIM uses a fixed kernel tiling scheme that produces per-core partition sizes that do not align with the 16-way SIMD instruction width, resulting in poor SIMD utilization. DCC addresses this by providing better kernel tiling configurations, improving performance by $2.26\times$ on average, and enabling HBM-PIM to outperform the GPU at tensor size 2048. Overall, DCC demonstrates robustness across multiple

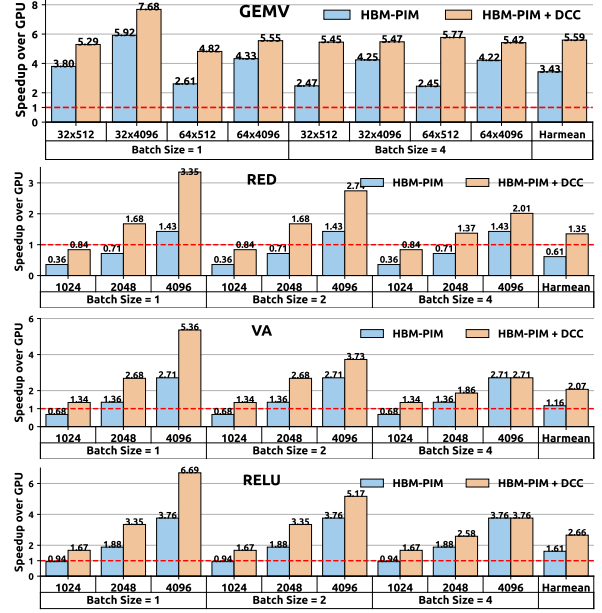


Fig. 8: Speedup of HBM-PIM and HBM-PIM+DCC over GPU for GEMV, RED, VA and RELU, varying the tensor sizes.

PIM backends through its multi-layer PIM abstraction, providing consistent performance improvements across diverse ML workloads on both HBM-PIM and AttAcc.

Fig. 9 shows the speedup of DCC over HBM-PIM and AttAcc, when increasing the batch size in GEMV, evaluating in an attention layer with head count 32, input size 128 and various output sizes. In both PIM backends, DCC improves performance over the baseline as the batch size increases. On average, DCC performance gains increase with batch size, from $1.33\times$ to $1.58\times$ on HBM-PIM, and from $1.14\times$ to $1.31\times$ on AttAcc, as batch size grows from 1 to 8. In large output dimension and batch sizes, AttAcc’s GEMV data distribution strategy achieves high performance, leaving limited room for further optimization. In those workloads, DCC is still able to *automatically* match and slightly improve AttAcc’s hand-optimized performance without programmer intervention, demonstrating that DCC can provide both high performance and high programming ease.

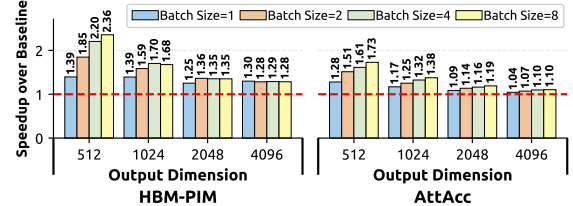


Fig. 9: Speedup of DCC over HBM-PIM (left) and AttAcc (right), when increasing the batch size in the GEMV kernel.

5.2. End-to-End LLM Inference

Fig. 10 presents the normalized execution time breakdown of AttAcc and AttAcc+DCC over GPU for the main computa-

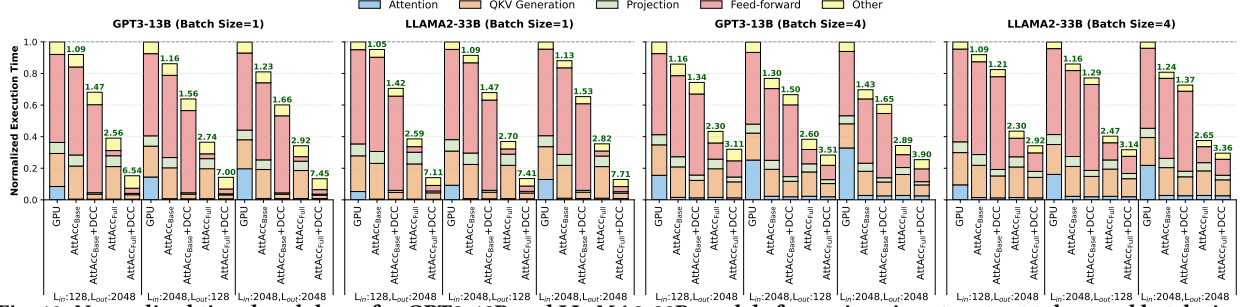


Fig. 10: Normalized time breakdown for GPT3-13B and LLaMA2-33B models for various input, output token and batch sizes.

tional phases in inference of two state-of-the-art models, while varying the input and output token sizes and batch sizes. We evaluate two variants: AttAcc_{Base} runs only attention layers on PIM, and AttAcc_{Full} runs attention layers and a portion of each Feed-forward layer on the PIM side (See [28]), while the largest portion of Feed-forward runs on GPU. For both AttAcc variants, QKV generation, projection and Other run on GPU. However, with DCC’s optimizations, we enable QKV generation and projection to also be executed on the PIM side, providing performance benefits over running them on GPU. In LLM inference, DCC initializes the kernels with several random requests, then generates tiling drafts on-the-fly for new tensor sizes; this generation time is included in our measurements. The numbers in the top of each bar show speedup over GPU.

We make three key observations. First, DCC provides high performance benefits on both AttAcc_{Base} and AttAcc_{Full} by $1.23\times$ and $1.75\times$ on average, respectively, improving performance over GPU by $1.44\times$ and $4.52\times$ on average, respectively. DCC significantly strengthens AttAcc, enabling it to substantially outperform GPU across different models and token sizes. Second, DCC improves performance on Attention layers by $1.12\times$ and $1.16\times$ on average over AttAcc for GPT3-13B and LLaMA2-33B, respectively. AttAcc uses a fixed tiling strategy across all token counts and batch sizes, while DCC adapts tiling strategies to different workload configurations, enabling these layers to run efficiently on PIM and significantly outperform GPU. Overall, we conclude that DCC provides significant performance benefits in various state-of-the-art LLMs with different token count and batch sizes. These results demonstrate that DCC can serve as a practical and effective compiler for heterogeneous ML acceleration on high-performance processors (e.g., GPUs) and PIM backends.

5.3. Large-Scale Experiments

Fig. 11 shows the normalized time breakdown of AttAcc variants and AttAcc+DCC schemes over the GPU baseline using MT-NLG-310B model, 128 input tokens, 2048 output tokens, FP16 and FP8 numerical precisions, and large batch sizes (up to 64). We use 8 A100 GPUs (DGX-class) with 5 HBM-type PIM devices in each GPU. DCC significantly improves performance over AttAcc_{Base} and AttAcc_{Full} by $1.59\times$ and $1.67\times$,

respectively. In large batch sizes, AttAcc’s GEMV data distribution (QKV generation and projection) already enables high performance. DCC improves it further by $2.21\times$, providing both high performance and high programming ease. DCC is highly efficient in larger batch sizes, larger LLMs, large multi-GPU systems and using different numerical precisions, providing robustness even in large-scale inference settings.

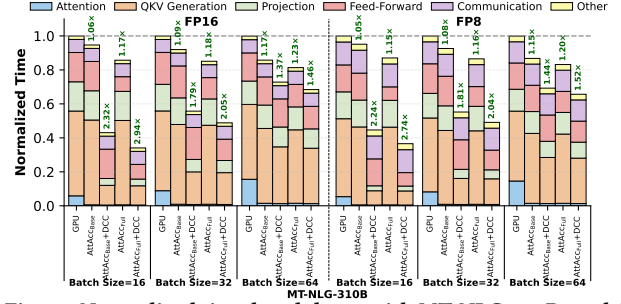


Fig. 11: Normalized time breakdown with MT-NLG-310B model for various batch sizes and numerical precisions using 8 A100 GPUs (DGX-class) with 5 PIM devices each GPU.

5.4. ML Kernel Time Breakdown

Fig. 12 shows the normalized time breakdown split into time spent on *compute* and *data rearrangements* for various ML kernels using two batch sizes, and two different backends with and without DCC. The numbers above bars show the speedup provided by DCC. The execution time is normalized to that of the respective PIM backend with its default implementation.

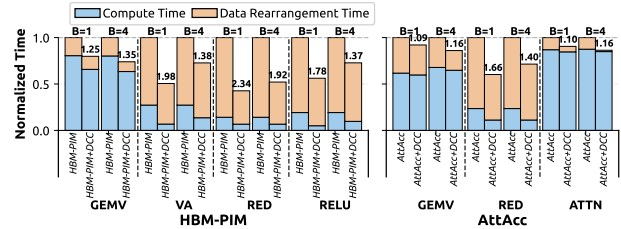


Fig. 12: Normalized time breakdown of compute and data rearrangement time in various ML workloads and backends.

We draw two conclusions. First, DCC co-optimizes both compute time and data rearrangement time, providing $1.58\times$ and $1.65\times$ speedup, respectively. DCC comprehensively explores a broader joint optimization space than the fixed tiling scheme used in HBM-PIM and AttAcc, and identifies highly efficient balance between data movement and computation costs. Second, DCC provides larger benefits in ML kernels, where data rearrangement dominates execution time. VA, RED, and RELU exhibit substantial data rearrangement overheads, and DCC accelerates them by $1.67\times$, while it accelerates the compute-

heavy GEMV and ATTN by $1.18\times$. These results show that DCC significantly alleviates data rearrangement costs in PIM, while enabling compute-specific optimizations.

5.5. Energy Consumption Analysis

Fig. 13 presents DCC’s in energy consumption as kernel breakdown using AttAcc_{Full} system compared to an A100 GPU in end-to-end inference of various LLMs, input token, output token, and batch sizes. We follow the methodology of the AttAcc paper [28]. AttAcc’s energy simulation synthesizes all arithmetic units using the Synopsys Design Compiler with a 7nm predictive PDK (ASAP7 [87]), models SRAM-based buffers using FinCACTI [88], and scales all components to the appropriate DRAM process technology node. Energy consumption inside the DRAM die is calculated using datapath lengths from HBM chip microphotographs [85,89]. DCC improves energy consumption over AttAcc_{Full} on average by $1.24\times$. DCC’s performance benefits and reduction in data movement translates to energy reduction.

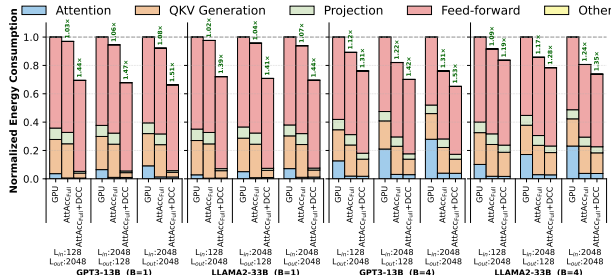


Fig. 13: Normalized energy breakdown in GPT3-13B and LLaMA2-33B comparing AttAcc_{Full} PIM system over GPU.

5.6. Compilation Time Evaluation

We evaluate DCC’s compilation time, and the effectiveness of its draft pruning scheme and its prediction model. Table 3 shows the DCC search space (#candidate drafts) and compilation time, when disabling versus enabling draft pruning for a single workload on AttAcc. We use 128 input tokens, 128 output tokens, 32 heads for ATTN or GEMV, and we vary the PIM devices per GPU and batch sizes.

Kernel	PIM devices per GPU	batch size	Number of GPUs	#drafts w/o pruning	#drafts w pruning	compilation time w/o pruning (s)	compilation time w pruning (s)
ATTN	5	4	1	32186	4274	9.57	2.96
ATTN	5	64	8	33271	4868	16.20	4.10
ATTN	10	4	1	79952	6200	29.90	4.22
ATTN	10	64	8	83708	7319	54.51	6.32
GEMV	5	4	1	32186	4274	19.75	3.75
GEMV	5	64	8	33271	4868	36.86	5.54
GEMV	10	4	1	79952	6200	57.66	5.46
GEMV	10	64	8	83708	7319	109.92	9.30

Table 3: DCC search space and compilation time, when disabling versus enabling the pruning for single kernel workload.

We draw three findings. First, DCC compilation time is low (a few seconds), occurring **only once** offline before inference. In real deployment scenarios with billions of inference requests on the same model and system configuration, the amortized compilation cost is negligible. Notably, compilation time of a single kernel does *not* significantly increase, when considering more tensor shapes: ATTN with 1 tensor shape takes 2.96s and with 272 different tensor shapes takes 4.74s (Table 4). Increasing tensor shapes during compilation primarily increases GPU utilization rather than compilation time. Second, the search space primarily grows with system scale, e.g., adding more PIM devices per GPU, as larger systems introduce more candidate data partitioning strategies. Increasing batch sizes (or

tensor shapes that we also tested) with a fixed number of PIM devices/cores only slightly increases the search space (the number of candidate partitioning strategies across cores remains similar). Third, our pruner significantly reduces the search space by on average $9.01\times$ and compilation time by $5.97\times$, with no impact on kernel execution performance. DCC’s pruning rules eliminate largely inefficient configurations, ensuring that DCC robustly finds near-optimal schedules.

We run ML kernels on HBM-PIM and AttAcc with hundreds of different tensor and batch sizes per kernel, representative configurations of real ML models, and all kernel configurations of all evaluated LLMs. We use DCC (pruning is disabled) to exhaustively generate all possible tiling drafts, execute all of them (the whole search space) on the target PIM backend (not using our predictor), and identify the true optimum draft. Table 4 compares the performance of the drafts selected by DCC’s coupled learning-based performance predictor against the performance of true optimum drafts. The *Total* column shows the number of tensor shapes and batch sizes configurations tested per kernel. The *#Best* column shows in how many cases our predictor correctly identified the true optimum draft. The *Compilation Time w/o Predictor* is the DCC compilation time when evaluating all candidate drafts by executing them on an A100 GPU to measure their performance (instead of using our coupled performance predictor). The *Compilation Time w Predictor* is the DCC compile time using our predictor. The *Suboptimal Kernel Performance* and *Suboptimal Total Performance* columns show the average performance degradation at kernel and end-to-end inference level, respectively, for the cases where the predictor selects a suboptimal schedule.

Backend-Workload	Total	#Best	Compilation Time w/o Predictor (s)	Compilation Time w Predictor (s)	Suboptimal Kernel Performance	Suboptimal Total Performance
AttAcc+ATTN	272	203	24.80	4.74	97.04% (69 cases)	-
AttAcc+GEMV	304	290	60.70	11.21	97.01% (14 cases)	-
AttAcc+RED	167	155	7.60	2.22	94.01% (12 cases)	-
AttAcc+GPT3	502	489	11.94	4.84	97.94% (13 cases)	99.89%
AttAcc+LLaMA2	502	488	16.13	5.49	98.83% (14 cases)	99.57%
AttAcc+MT-NLG	251	245	56.53	11.22	99.07% (6 cases)	99.80%
HBM-PIM+GEMV	304	265	116.07	19.50	95.84% (39 cases)	-
HBM-PIM+RED	167	154	8.09	2.29	96.43% (13 cases)	-
HBM-PIM+VA	167	158	14.21	3.24	94.94% (9 cases)	-
HBM-PIM+RELU	167	157	5.76	1.94	94.48% (10 cases)	-

Table 4: Prediction accuracy, compilation time and suboptimal performance using DCC across various ML workloads.

DCC’s coupled predictor provides on average 89.28% and 97.37% accuracy on single kernels and LLMs, respectively, in correctly identifying the true optimum schedules. When suboptimal, the predictor’s selections provide 96.56% of true optimal performance on average across different workloads and backends. In AttAcc+ATTN, the DCC predictor has relatively lower accuracy (74.63%), since ATTN fuses GEMV and softmax as a single kernel. However, it still provides high performance (97.4% of true optimum). Even in large LLM inference, DCC’s predictor rarely finds sub-optimal schedules, having only $\sim 0.2\%$ total performance slowdown over when using the true optimum schedules. Moreover, our predictor significantly accelerates compilation time by $3.81\times$ averaged across all workloads, LLMs, and backends. Overall, our results show that even when our predictor fails to identify the true optimum, the configuration it selects has negligible performance degradation, while significantly reducing the compilation time.

5.7. Dynamic Prediction Evaluation

We evaluate DCC’s compilation overhead in an online draft selection scenario, where the kernel workload is *unseen* and DCC

has not yet recorded the best-performing draft configuration in its lookup table. This scenario requires DCC to search and estimate the best-performing configuration at runtime. We execute 2000 batches (batch size 64) of synthetic inference queries on MT-NLG-310B, with input and output token sizes randomly generated from the range [128, 2048], covering representative sequence lengths of real-world deployments. Fig. 14 compares *online DCC* with unseen tensor shapes against *offline DCC*, where all tensor shapes have been previously seen and are recorded in the lookup table. Online DCC incurs performance penalty only during the first ~ 160 queries, where unseen tensor shapes trigger additional compilation. After this short warmup period, DCC’s lookup table covers all encountered tensor shapes and online DCC converges to the same performance as offline DCC. DCC’s online overhead is short-lived, requiring only a few hundred warmup queries, and is fully amortized in real-world LLM deployments where the number of requests (typically millions of LLM queries) is orders of magnitude larger than DCC’s warmup period.

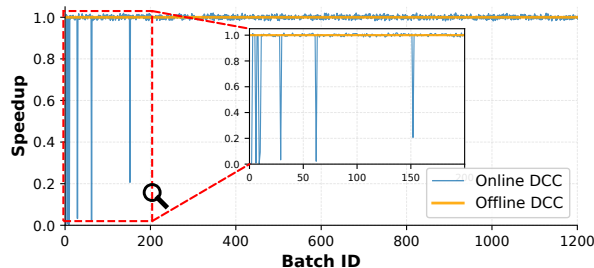


Fig. 14: Online vs offline DCC on MT-NLG 310B inference.

6. Related Work

To our knowledge, this is the first work that (i) comprehensively considers data rearrangement strategies and their associated costs in the ML kernel tuning process for PIM devices, (ii) proposes a data-centric compiler that *jointly optimizes* data rearrangement strategies and compute schedules, and (iii) supports multiple PIM backends. We briefly discuss prior work.

Compiler Support for PIM. Prior works [21,52,54,55] design compilation tools for PIM systems, but lack systematic optimization and auto-tuning capabilities. They target a single ML kernel or a single PIM backend. ATiM [42] is a search-based tensor compiler for UPMEM PIM. However, UPMEM’s DDR4-based architecture targets CPU memory channels, thus preventing GPU-PIM co-executions for ML models. ATiM employs *compute-centric* tuning, and such an approach yields sub-optimal performance (See §2.2). In contrast, DCC has a *data-centric* approach to minimize end-to-end kernel time.

Compiler Support for Processing-Using-Memory (PUM). Prior works [90–94] propose compilers for PUM devices, which are analog-based and have a higher hardware design complexity than PIM devices (digital-based).

PIM Architectures and Accelerators. UPMEM PIM [19,24] DDR4-based device for CPUs lacks a complete 32×32 -bit integer multiplier and floating-point units, making it unsuitable for our target ML workloads [18–20,95–97]. Samsung HBM-PIM [25] and SK Hynix GDDR6-AiM [26,27] can be integrated with GPUs. Numerous prior works [18–23,28–38,42–49] enhance near-bank PIM devices to support critical ML primitives

(e.g., GEMV, ReLU, Softmax), and DCC can be directly used on them. Near-rank PIM systems [53,98–101] place cores at the buffer chip of the DIMM with access to all banks. Despite different core placement, DCC can effectively support near-rank PIM designs for ML. DCC can be easily extended to support hybrid near-rank and near-bank designs [53,102–104] and NPU-integrated PIM devices [105–110] thanks to its multi-layer abstraction. Prior works [50,99,111–127] design application-specific PIM devices for graph analytics or data retrieval and analytics. While DCC may benefit some kernels in these devices, we primarily target ML kernels.

Software for PIM Systems. Prior works [18–20,51,95–97,128–140] propose libraries, frameworks, and benchmark suites only for UPMEM spanning various application domains. UPMEM PIM is not well-suited for running ML kernels. DCC targets PIM devices that efficiently support ML workloads.

Communication and System Integration for PIM. Prior works [57,141–152] design efficient data transfers, memory management, synchronization, virtualization of PIM, and simulation tools. PIMCARE [57] is a compiler-assisted scheduler to allocate the correct amount of PIM devices for an application. UniNDP [56] is a simulation and compilation framework for different PIM architecture types (near-bank, near-device, near-rank, near-channel) to guide programmers how to develop their kernels. To run a kernel on a target PIM backend, programmers still need to manually implement it using backend-specific low-level programming interface. DCC directly takes kernels written in high-level Python interface (§4.5), and automatically generates optimized low-level code for a PIM backend.

ML Compilers for Commodity Systems. Prior works [61–73,153–155] propose ML compilers for CPU, FPGA and GPU systems by leveraging their shared memory model and deep cache hierarchies (on-chip caches). Prometheus [155] optimizes kernels on FPGAs via loop transformations for cache locality, and overlapping computation with off-chip data transfers to main memory. Instead, PIM systems have a distributed memory model and shallow cache hierarchy, and may not have hardware support to overlap PIM computation with off-chip data transfers. Thus, ML compilers for CPU/GPU/FPGAs cannot be directly used in PIM systems, or would cause high performance costs because they do not optimize how data is distributed across DRAM banks.

7. Conclusion

We introduce DCC, the first data-centric compiler for ML kernels on PIM devices, supporting multiple PIM backends. The key idea of DCC is to jointly optimize data rearrangement strategies and compute schedules in a unified tuning process, minimizing end-to-end kernel execution time. DCC provides up to $7.68 \times$ speedup over GPU on HBM-PIM and up to $13.17 \times$ speedup over GPU on AttAcc for ML kernels. DCC on AttAcc accelerates end-to-end LLM inference by up to $7.71 \times$. We hope our work encourages further PIM compilation studies.

Acknowledgements

We thank the anonymous reviewers of ISCA 2026 for valuable feedback. We thank Qidong Su, Zhanda Zhu and Yaoyao Ding for their suggestions. We thank the SPIN Research Group, the SAFARI Research Group and the EcoSystem Research Group for providing a stimulating intellectual environment.

References

- [1] J. B. Heaton, N. G. Polson, and J. H. Witte, "Deep Learning for Finance: Deep Portfolios," *Appl. Stoch. Model. Bus. Ind.*, 2017.
- [2] A. M. Ozbayoglu, M. U. Gudelek, and O. B. Sezer, "Deep Learning for Financial Applications: A Survey," *Appl. Soft Comput.*, 2020.
- [3] K. Oosthuizen, E. Botha, J. Robertson, and M. Montecchi, "Artificial Intelligence in Retail: The AI-Enabled Value Chain," *Australasian Marketing Journal*, 2021.
- [4] D. Ravi, C. Wong, F. Deligianni, M. Berthelot, J. Andreu-Perez, B. Lo, and G.-Z. Yang, "Deep Learning for Health Informatics," *IEEE J. Biomed. Health Inform.*, 2016.
- [5] R. Miotto, F. Wang, S. Wang, X. Jiang, and J. T. Dudley, "Deep Learning for Healthcare: Review, Opportunities and Challenges," *Brief Bioinform.*, 2018.
- [6] R. S. Baker and P. S. Inventado, *Educational Data Mining and Learning Analytics*. Springer New York, 2014.
- [7] V. Zhang, B. Jeffries, and I. Koprinska, "A Machine Learning Approach for Predicting Student Progress in Online Programming Education," *Int. J. Artif. Intell. Educ.*, 2025.
- [8] K. Ota, M. S. Dao, V. Mezaris, and F. G. D. Natale, "Deep Learning for Mobile Multimedia: A Survey," *TOMM*, 2017.
- [9] Y. Zheng, "Predicting Movie Box Office Based on Machine Learning, Deep Learning, and Statistical Methods," *Appl. Comput. Eng.*, 2024.
- [10] Y. Tang, C. Zhao, J. Wang, C. Zhang, Q. Sun, W. X. Zheng, W. Du, F. Qian, and J. Kurths, "Perception and Navigation in Autonomous Systems in the Era of Learning: A Survey," *IEEE TNNLS*, 2022.
- [11] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The Cityscapes Dataset for Semantic Urban Scene Understanding," in *CVPR*, 2016.
- [12] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving," in *OSDI*, 2024.
- [13] Q. Su, W. Zhao, X. Li, M. Andoorveedu, C. Jiang, Z. Zhu, K. Song, C. Giannoula, and G. Pekhimenko, "Seesaw: High-Throughput LLM Inference via Model Re-Sharding," *MLSys*, 2026.
- [14] M. Kim, S. Hong, R. Ko, S. Choi, H. Lee, J. Kim, J.-Y. Kim, and J. Park, "Oaken: Fast and Efficient LLM Serving with Online-Offline Hybrid KV Cache Quantization," in *ISCA*, 2025.
- [15] A. K. Kamath, R. Prabhu, J. Mohan, S. Peter, R. Ramjee, and A. Panwar, "POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference," in *ASPLOS*, 2025.
- [16] C. Li and Y. Xu, "Foreseer: Knowledge-Driven Acceleration of Memory-Bound Matrix Multiplications for Large Language Model Inference," in *SYSTOR*, 2024.
- [17] X. Jiang, Y. Zhou, S. Cao, I. Stoica, and M. Yu, "NEO: Saving GPU Memory Crisis with CPU Offloading for Online LLM Inference," in *MLSys*, 2024.
- [18] J. Gómez-Luna, Y. Guo, S. Brocard, J. Legriel, R. Cimdado, G. F. Oliveira, G. Singh, and O. Mutlu, "Evaluating Machine Learning Workloads on Memory-Centric Computing Systems," in *ISPASS*, 2023.
- [19] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System," *IEEE Access*, 2022.
- [20] C. Giannoula, P. Yang, I. Fernandez, J. Yang, S. Durvasula, Y. X. Li, M. Sadrosadati, J. G. Luna, O. Mutlu, and G. Pekhimenko, "PyGim: An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures," *POMACS*, 2025.
- [21] C. Li, Z. Zhou, Y. Wang, F. Yang, T. Cao, M. Yang, Y. Liang, and G. Sun, "PIM-DL: Expanding the Applicability of Commodity DRAM-PIMs for Deep Learning via Algorithm-System Co-Optimization," in *ASPLOS*, 2024.
- [22] A. Boroumand, S. Ghose, B. Akin, R. Narayanaswami, G. F. Oliveira, X. Ma, E. Shiu, and O. Mutlu, "Google Neural Network Models for Edge Devices: Analyzing and Mitigating Machine Learning Inference Bottlenecks," in *PACT*, 2021.
- [23] A. Boroumand, S. Ghose, Y. Kim, R. Ausavarungrinun, E. Shiu, R. Thakur, D. Kim, A. Kuusela, A. Knies, P. Ranganathan, and O. Mutlu, "Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks," in *ASPLOS*, 2018.
- [24] F. Devaux, "The True Processing In Memory Accelerator," in *HCS*, 2019.
- [25] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product," in *ISCA*, 2021.
- [26] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim *et al.*, "A 1nm 1.25 V 8GB, 16GB/s/Pin GDDR6-Based Accelerator-in-Memory Supporting 1Tflops MAC Operation and Various Activation Functions for Deep Learning Applications," in *ISSCC*, 2022.
- [27] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, "Newton: A DRAM-Maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning," in *MICRO*, 2020.
- [28] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! Unleashing the Power of PIM for Batched Transformer-Based Generative Model Inference," in *ASPLOS*, 2024.
- [29] B. Kim, C. Lee, G. Kim, and E. Park, "Cost-Effective Extension of DRAM-PIM for Group-Wise LLM Quantization," *IEEE*, 2025.
- [30] Z. Li, J. Zhou, X. Li, and N. Sun, "BlockPIM: Optimizing Memory Management for PIM-Enabled Long-Context LLM Inference," in *DAC*, 2025.
- [31] S. Liu, Z. Huang, J. Yu, Q. Liu, and C. Chen, "McPAL: Scaling Unstructured Sparse Inference with Multi-Chiplet HBM-PIM Architecture for LLMs," in *DAC*, 2025.
- [32] L. Chen, D. Lyu, Z. Li, J. Jiang, Q. Wang, Z. Mao, and N. Jing, "AttenPIM: Accelerating LLM Attention with Dual-Mode GEMV in Processing-in-Memory," in *DAC*, 2025.
- [33] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "Papi: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-in-Memory-Enabled Computing System," in *ASPLOS*, 2025.
- [34] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, "SpecPIM: Accelerating Speculative Inference on PIM-Enabled System via Architecture-Dataflow Co-Exploration," in *ASPLOS*, 2024.
- [35] Y. Wu, Z. Wang, and W. D. Lu, "PIM GPT: A Hybrid Process in Memory Accelerator for Autoregressive Transformers," *npj Unconventional Computing*, 2024.
- [36] H. Lee, D. Baek, J. Son, J. Choi, K. Moon, and M. Jang, "PAISE: PIM-Accelerated Inference Scheduling Engine for Transformer-Based LLM," in *HPCA*, 2025.
- [37] W. Kim, Y. Lee, Y. Kim, J. Hwang, S. Oh, J. Jung, A. Huseynov, W. G. Park, C. H. Park, D. Mahajan, and J. Park, "Pimba: A Processing-in-Memory Acceleration for Post-Transformer Large Language Model Serving," in *MICRO*, 2025.
- [38] D. Quinn, E. E. Yücel, J. Kim, J. F. Martínez, and M. Alian, "LongSight: Compute-Enabled Memory to Accelerate Large-Context LLMs via Sparse Attention," in *MICRO*, 2025.
- [39] J. Gómez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking Memory-Centric Computing Systems: Analysis of Real Processing-in-Memory Hardware," in *IGSC*, 2021.
- [40] J. Gómez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking Memory-Centric Computing Systems: Analysis of Real Processing-in-Memory Hardware," in *CUT Workshop*, 2021.
- [41] D. Niu, S. Li, Y. Wang, W. Han, Z. Zhang, Y. Guan, T. Guan, F. Sun, F. Xue, L. Duan *et al.*, "184QPS/W 64Mb/mm2 3D Logic-to-DRAM Hybrid Bonding With Process-Near-Memory Engine for Recommendation System," in *ISSCC*, 2022.
- [42] Y. Shin, D. Kang, and H. Sung, "ATiM: Autotuning Tensor Programs for Processing-in-DRAM," in *ISCA*, 2025.
- [43] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference," in *ASPLOS*, 2025.
- [44] H. Shin, D. Kim, E. Park, S. Park, Y. Park, and S. Yoo, "McDRAM: Low Latency and Energy-Efficient Matrix Computations in DRAM," *TCAD*, 2018.
- [45] M. A. Alves, P. C. Santos, F. B. Moreira, M. Diener, and L. Carro, "Saving Memory Movements Through Vector Processing in the DRAM," in *CASES*, 2015.
- [46] E. Lockerman, A. Feldmann, M. Bakhshalipour, A. Stanescu, S. Gupta, D. Sanchez, and N. Beckmann, "Livia: Data-Centric Computing Throughout the Memory Hierarchy," in *ASPLOS*, 2020.
- [47] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, "Tetris: Scalable and Efficient Neural Network Acceleration With 3D Memory," in *ASPLOS*, 2017.
- [48] L. Ke, X. Zhang, J. So, J.-G. Lee, S.-H. Kang, S. Lee, S. Han, Y. Cho, J. H. Kim, Y. Kwon *et al.*, "Near-Memory Processing in Action: Accelerating Personalized Recommendation With AXDIMM," *MICRO*, 2021.
- [49] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungrinun, "Processing Data Where It Makes Sense: Enabling In-Memory Computation," *Microprocessors and Microsystems*, 2019.
- [50] C. Giannoula, I. Fernandez, J. G. Luna, N. Koziris, G. Goumas, and O. Mutlu, "SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-in-Memory Architectures," in *POMACS*, 2022.
- [51] S. Rhyner, H. Luo, J. Gómez-Luna, M. Sadrosadati, J. Jiang, A. Olgun, H. Gupta, C. Zhang, and O. Mutlu, "Analysis of Distributed Optimization Algorithms on a Real Processing-In-Memory System," in *PACT*, 2024.
- [52] J. Chen, J. Gómez-Luna, I. E. Hajj, Y. Guo, and O. Mutlu, "SimplePIM: A Software Framework for Productive and Efficient Programming of Real PIM Systems," in *PACT*, 2023.
- [53] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference," in *ASPLOS*, 2025.
- [54] A. A. Khan, H. Farzaneh, K. F. A. Friebe, C. Fournier, L. Chelini, and J. Castrillon, "Cinn (Cinnamon): A Compilation Infrastructure for Heterogeneous Compute In-Memory and Compute Near-Memory Paradigms," in *ASPLOS*, 2024.
- [55] Y. Shin, J. Park, S. Cho, and H. Sung, "PIMFlow: Compiler and Runtime Support for CNN Models on Processing-in-Memory DRAM," in *CGO*, 2023.
- [56] T. Xie, Z. Zhu, B. Li, Y. He, C. Li, G. Sun, H. Yang, Y. Xie, and Y. Wang, "UniNDP: A Unified Compilation and Simulation Tool for Near DRAM Processing Architectures," in *HPCA*, 2025.
- [57] I. Hwang, D. Kim, S. Kang, T. Park, T. Kim, J. Seo, H. Kim, Y. Kim, and Y. Park, "PIM-CARE: A Compiler-Assisted Dynamic Resource Allocation Framework for Real-World DRAM PIM," in *ICS*, 2025.
- [58] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungrinun, "A Modern Primer on Processing in Memory," in *Emerging Computing: From Devices to Systems - Looking Beyond Moore and Von Neumann*, 2021.
- [59] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungrinun, "Enabling Practical Processing In and Near Memory for Data-Intensive Computing," in *DAC*, 2019.
- [60] O. Mutlu, A. Olgun, and I. E. Yuksel, "Memory-Centric Computing: Solving Computing's Memory Problem," in *IMW*, 2025.
- [61] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," in *OSDI*, 2018.
- [62] S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu *et al.*, "TensorIR: An Abstraction for Automatic Tensorized Program Optimization," in *ASPLOS*, 2023.
- [63] Y. Ding, C. H. Yu, B. Zheng, Y. Liu, Y. Wang, and G. Pekhimenko, "Hidet: Task-Mapping Programming Paradigm for Deep Learning Tensor Programs," in *ASPLOS*, 2023.
- [64] J. Xing, L. Wang, S. Zhang, J. Chen, A. Chen, and Y. Zhu, "Bolt: Bridging the Gap Between Auto-Tuners and Hardware-Native Performance," *MLSys*, 2022.
- [65] A. Gupta, Y. Yuan, D. Jain, Y. Ge, D. Aponte, Y. Zhou, and C. Mendis, "SPAT: A Framework for Optimised GPU Code-Generation for SParse regular Attention," *Proc. ACM Program. Lang.*, 2025.

- [66] F. Liu, S. Huang, N. Yang, Z. Wang, H. Li, and L. Jiang, "CROSS: Compiler-Driven Optimization of Sparse DNNs Using Sparse/Dense Computation Kernels," in *HPCA*, 2025.
- [67] W. Ahrens, T. F. Collin, R. Patel, K. Deeds, C. Hong, and S. Amarasinghe, "Finch: Sparse and Structured Tensor Programming with Control Flow," *Proc. ACM Program. Lang.*, 2025.
- [68] Z. Du, Y. Liu, N. Sun, H. Cui, X. Feng, and J. Li, "SRsparse: Generating Codes for High-Performance Sparse Matrix-Vector Semiring Computations," *ACM Trans. Archit. Code Optim.*, 2025.
- [69] J. Won, C. Hong, C. Mendis, J. Emer, and S. Amarasinghe, "Unified Convolution Framework: A Compiler-Based Approach to Support Sparse Convolutions," *MLSys*, 2023.
- [70] Z. Ye, R. Lai, J. Shao, T. Chen, and L. Ceze, "SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning," in *ASPLOS*, 2023.
- [71] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, "The Tensor Algebra Compiler," *Proc. ACM Program. Lang.*, 2017.
- [72] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen *et al.*, "Ansor: Generating High-Performance Tensor Programs for Deep Learning," in *OSDI*, 2020.
- [73] T. Chen, L. Zheng, E. Yan, Z. Jiang, T. Moreau, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Learning to Optimize Tensor Programs," *NeurIPS*, 2018.
- [74] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *KDD*, 2016.
- [75] J. Shao, X. Zhou, S. Feng, B. Hou, R. Lai, H. Jin, W. Lin, M. Masuda, C. H. Yu, and T. Chen, "Tensor Program Optimization with Probabilistic Programs," *NeurIPS*, 2022.
- [76] V. Ganesan, S. Selvam, S. Sen, P. Kumar, and A. Raghunathan, "A Case for Generalizable DNN Cost Models for Mobile Devices," in *ISWC*, 2020.
- [77] B. H. Ahn, P. Pilligundla, A. Yazdankhsh, and H. Esmaeilzadeh, "Chameleon: Adaptive Code Optimization for Expedited Deep Neural Network Compilation," *ICLR*, 2020.
- [78] A. Mishra, S. Chheda, C. Soto, A. M. Malik, M. Lin, and B. Chapman, "Compoff: A Compiler Cost Model Using Machine Learning to Predict the Cost of OpenMP Offloading," in *IPDPSW*, 2022.
- [79] Python Software Foundation, "Python language reference, version 3.x," 2016. [Online]. Available: <https://www.python.org>
- [80] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *NeurIPS*, 2019.
- [81] H. Luo, Y. C. Tuğrul, F. Bostanci, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator," *arXiv preprint arXiv:2308.11030*, 2023.
- [82] CMU-SAFARI, "Ramulator2: A Fast and Extensible DRAM Simulator," <https://github.com/CMU-SAFARI/ramulator2>, 2026.
- [83] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A Fast and Extensible DRAM Simulator," *IEEE CAL*, 2016.
- [84] CMU-SAFARI, "Ramulator: A DRAM Simulator," <https://github.com/CMU-SAFARI/ramulator>, 2021.
- [85] Y. Ryu, S.-G. Ahn, J. H. Lee, J. Park, Y. K. Kim, H. Kim, Y. G. Song, H.-W. Cho, S. Cho, S.-H. Song, H. Lee, U. Shin, J. Ahn, J.-M. Ryu, S. Lee, K.-H. Lim, J. Lee, J. H. Park, J.-S. Jeong, S. Joo, D. Cho, S. Y. Kim, M. Lee, H. Kim, M. Kim, J.-S. Kim, J. Kim, H. G. Kang, M.-K. Lee, S.-R. Kim, Y.-C. Kwon, Y. Y. Byun, K. Lee, S. Park, J. Youn, M.-O. Kim, K. Sohn, S.-J. Hwang, and J. Lee, "A 16 GB 1024 GB/s HBM3 DRAM With Source-Synchronized Bus Design and On-Die Error Control Scheme for Enhanced RAS Features," *IEEE Journal of Solid-State Circuits*, 2023.
- [86] S. Gurumurthi, K. Lee, M. Jang, V. Sridharan, A. Nygren, Y. Ryu, K. Sohn, T. Kim, and H. Chung, "HBM3: Enabling Memory Resilience at Scale," *IEEE CAL*, 2021.
- [87] L. T. Clark, V. Vashishtha, L. Shiffren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "ASAP7: A 7-nm FinFET Predictive Process Design Kit," *Microelectronics Journal*, 2016.
- [88] A. Shafaei, Y. Wang, X. Lin, and M. Pedram, "FinCACTI: Architectural Analysis and Modeling of Caches with Deeply-Scaled FinFET Devices," in *ISVLSI*, 2014.
- [89] M.-J. Park, H. S. Cho, T.-S. Yun, S. Byeon, Y. J. Koo, S. Yoon, D. U. Lee, S. Choi, J. Park, J. Lee, K. Cho, J. Moon, B.-K. Yoon, Y.-J. Park, S.-m. Oh, C. K. Lee, T.-K. Kim, S.-H. Lee, H.-W. Kim, Y. Ju, S.-K. Lim, S. G. Baek, K. Y. Lee, S. H. Lee, W. S. We, S. Kim, Y. Choi, S.-H. Lee, S. M. Yang, G. Lee, I.-K. Kim, Y. Jeon, J.-H. Park, J. C. Yun, C. Park, S.-Y. Kim, S. Kim, D.-Y. Lee, S.-H. Oh, T. Hwang, J. Shin, Y. Lee, H. Kim, J. Lee, Y. Hur, S. Lee, J. Jang, J. Chun, and J. Cho, "A 192-Gb 12-High 896-GB/s HBM3 DRAM with a TSV Auto-Calibration Scheme and Machine-Learning-Based Layout Optimization," in *ISSCC*, 2022.
- [90] J. Liu, M. Zhou, Y. Pan, C.-Y. Yang, L. Josipović, and T. Rosing, "OptiPIM: Optimizing Processing-in-Memory Acceleration Using Integer Linear Programming," in *ISCA*, 2025.
- [91] G. Oliveira, A. Olgun, A. G. Yağlıkçı, F. N. Bostanci, J. Gómez-Luna, S. Ghose, and O. Mutlu, "MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Computing," in *HPCA*, 2024.
- [92] A. Khadem, D. Fujiki, H. Chen, Y. Gu, N. Talati, S. Mahlke, and R. Das, "Multi-Dimensional Vector ISA Extension for Mobile In-Cache Computing," in *HPCA*, 2025.
- [93] A. Drebes, L. Chelini, O. Zinenko, A. Cohen, H. Corporaal, T. Grosser, K. Vadiel, and N. Vasilache, "TC-CIM: Empowering Tensor Comprehensions for Computing-In-Memory," in *IMPACT*, 2020.
- [94] X. Peng, Y. Wang, and M.-C. Yang, "CHOPPER: A Compiler Infrastructure for Programmable Bit-serial SIMD Processing Using Memory in DRAM," in *HPCA*, 2023.
- [95] C. Giannoula, I. Fernandez, J. G. Luna, N. Koziris, G. Goumas, and O. Mutlu, "SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures," *POMACS*, 2022.
- [96] C. Giannoula, I. Fernandez, J. Gómez-Luna, N. Koziris, G. Goumas, and O. Mutlu, "Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures," in *SIGMETRICS*, 2022.
- [97] C. Giannoula, P. Yang, I. F. Vega, J. Yang, Y. X. Li, J. G. Luna, M. Sadrosadati, O. Mutlu, and G. Pekhimenko, "Accelerating Graph Neural Networks on Real Processing-In-Memory Systems," *arXiv*, 2024.
- [98] M. Rhee, J. Sim, T. Ahn, S. Lee, D. Yoon, E. Kim, K. Park, Y. Joo, and H. Kim, "HPU: High-Bandwidth Processing Unit for Scalable, Cost-Effective LLM Inference via GPU Co-Processing," *arXiv preprint arXiv:2504.16112*, 2025.
- [99] Y. Kwon, Y. Lee, and M. Rhu, "TensorDimm: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning," in *MICRO*, 2019.
- [100] L. Liu, S. Zhao, B. Li, H. Ren, Z. Xu, M. Wang, X. Li, Y. Han, and Y. Wang, "Make LLM Inference Affordable to Everyone: Augmenting GPU Memory with NDP-DIMM," in *HPCA*, 2025.
- [101] D. Lee, J. So, M. AHN, J.-G. Lee, J. Kim, J. Cho, R. Oliver, V. C. Thummala, R. s. JV, S. S. Upadhyay, M. I. Khan, and J. H. Kim, "Improving In-Memory Database Operations with Acceleration DIMM (AxDIMM)," in *DaMoN*, 2022.
- [102] J.-W. Jang, J. Oh, Y. Kong, J.-Y. Hong, S.-H. Cho, J. Lee, H. Yang, and J.-S. Yang, "Accelerating Retrieval Augmented Language Model via PIM and PNM Integration," in *MICRO*, 2025.
- [103] D. Quinn, E. E. Yücel, M. Prammer, Z. Fan, K. Skadron, J. M. Patel, J. F. Martinez, and M. Alian, "DReX: Accurate and Scalable Dense Retrieval Acceleration via Algorithmic-Hardware Codesign," in *ISCA*, 2025.
- [104] D. Kim, J.-Y. Kim, W. Han, J. Won, H. Choi, Y. Kwon, and J.-Y. Kim, "Darwin: A DRAM-Based Multi-Level Processing-in-Memory Architecture for Column-Oriented Database," *IEEE Trans. Emerging Top. Comput.*, 2024.
- [105] S. He, Z. Zhu, Y. He, and T. Jia, "LP-Spec: Leveraging LPDDR PIM for Efficient LLM Mobile Speculative Inference with Architecture-Dataflow Co-Optimization," *arXiv*, 2025.
- [106] S. Han, B. Yoon, G. Park, C. Song, D. Kim, and J.-J. Kim, "Near-Memory LLM Inference Processor Based on 3D DRAM-to-Logic Hybrid Bonding," in *DAC*, 2025.
- [107] L. Wu, H. Zhu, S. He, X. Lin, X. Zeng, and C. Chen, "PIMoE: Towards Efficient MoE Transformer Deployment on NPU-PIM System Through Throttle-Aware Task Offloading," in *DAC*, 2025.
- [108] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin *et al.*, "Janus: Integrated Accelerator Based on NPU-PIM Unified Memory System," in *ASPLOS*, 2024.
- [109] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMS: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing," in *ASPLOS*, 2024.
- [110] R. Chen, Z. Song, Y. Zheng, Z. Zhu, G. Li, N. Jing, X. Liang, and H. Guan, "HEAT: NPU-NDP Heterogeneous Architecture for Transformer-Empowered Graph Neural Networks," in *MICRO*, 2025.
- [111] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing," in *ISCA*, 2015.
- [112] G. Dai, T. Huang, Y. Chi, J. Zhao, G. Sun, Y. Liu, Y. Wang, Y. Xie, and H. Yang, "GraphH: A Processing-in-Memory Architecture for Large-Scale Graph Processing," *IEEE TCAD*, 2018.
- [113] T. Kang, G. Choi, T. Suh, and G. Koo, "SparsePIM: An Efficient HBM-Based PIM Architecture for Sparse Matrix-Vector Multiplications," in *ICS*, 2025.
- [114] D. Lee, B. Hyun, T. Kim, and M. Rhu, "PIM-MMU: A Memory Management Unit for Accelerating Data Transfers in Commercial PIM Systems," in *MICRO*, 2024.
- [115] Z. Zhou, C. Li, X. Wei, X. Wang, and G. Sun, "Gnnear: Accelerating Full-Batch Training of Graph Neural Networks with Near-Memory Processing," in *PACT*, 2022.
- [116] T. Tian, X. Wang, L. Zhao, W. Wu, X. Zhang, F. Lu, T. Wang, and X. Jin, "G-NMP: Accelerating Graph Neural Networks with DIMM-Based Near-Memory Processing," *Journal of Systems Architecture*, 2022.
- [117] S. Yun, H. Nam, J. Park, B. Kim, J. H. Ahn, and E. Lee, "GraNDe: Efficient Near-Data Processing Architecture for Graph Neural Networks," *IEEE Transactions on Computers*, 2023.
- [118] D. Chen, H. He, H. Jin, L. Zheng, Y. Huang, X. Shen, and X. Liao, "MetaNMP: Leveraging Cartesian-Like Product to Accelerate HGNs with Near-Memory Processing," in *ISCA*, 2023.
- [119] M. Saeed, P. J. Nair, and T. M. Aamodt, "RayN: Ray Tracing Acceleration with Near-Memory Computing," in *MICRO*, 2025.
- [120] L. Yan, M. Zhang, R. Wang, X. Chen, X. Zou, X. Lu, Y. Han, and X.-H. Sun, "CoPIM: A Concurrency-Aware PIM Workload Offloading Architecture for Graph Applications," in *ISLPED*, 2021.
- [121] M. Besta, R. Kanakagiri, G. Kwasniewski, R. Ausavarungrunirun, J. Beránek, K. Kanellopoulos, K. Janda, Z. Vónarburg-Shmaria, L. Gianinazzi, I. Stefan *et al.*, "Sisa: Set-Centric Instruction Set Architecture for Graph Mining on Processing-In-Memory Systems," in *MICRO*, 2021.
- [122] L. Nai, R. Hadidi, J. Sim, H. Kim, P. Kumar, and H. Kim, "Graphpim: Enabling Instruction-Level PIM Offloading in Graph Computing Frameworks," in *HPCA*, 2017.
- [123] A. Boroumand, S. Ghose, G. F. Oliveira, and O. Mutlu, "Polynesia: Enabling High-Performance and Energy-Efficient Hybrid Transactional/Analytical Databases with Hardware/Software Co-Design," in *ICDE*, 2022.
- [124] J. Kim, D. Senol Cali, H. Xin, D. Lee, S. Ghose, M. Alser, H. Hassan, O. Ergin, C. Alkan, and O. Mutlu, "GRIM-Filter: Fast Seed Location Filtering in DNA Read Mapping Using Processing-in-Memory Technologies," *BMC Genomics*, 2018.

- [125] M. Drumond, A. Daglis, N. Mirzadeh, D. Ustiugov, J. Picorel, B. Falsafi, B. Grot, and D. Pnevmatikatos, "The Mondrian Data Engine," in *ISCA*, 2017.
- [126] D. S. Cali, G. S. Kalsi, Z. Bingöl, C. Firtina, L. Subramanian, J. S. Kim, R. Ausavarungnirun, M. Alser, J. Gomez-Luna, A. Boroumand, A. Norion, A. Scibisz, S. Subramoneyon, C. Alkan, S. Ghose, and O. Mutlu, "GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis," in *MICRO*, 2020.
- [127] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-Enabled Instructions: A Low-overhead, Locality-Aware Processing-in-Memory Architecture," in *ISCA*, 2015.
- [128] S. Diab, A. Nassereldine, M. Alser, J. Gómez Luna, O. Mutlu, and I. El Hajj, "A Framework for High-Throughput Sequence Alignment Using Real Processing-in-Memory Systems," in *Bioinformatics*, 2023.
- [129] C. Lim, S. Lee, J. Choi, J. Lee, S. Park, H. Kim, J. Lee, and Y. Kim, "Design and Analysis of a Processing-in-DIMM Join Algorithm: A Case Study with UPMEM DIMMs," in *Proc. ACM Manag. Data*, 2023.
- [130] M. Item, G. F. Oliveira, J. Gómez-Luna, M. Sadrosadati, Y. Guo, and O. Mutlu, "TransPimLib: Efficient Transcendental Functions for Processing-in-Memory Systems," in *ISPASS*, 2023.
- [131] P. Das, P. R. Sutradhar, M. Indovina, S. M. P. Dinakarrao, and A. Ganguly, "Implementation and Evaluation of Deep Neural Networks in Commercially Available Processing in Memory Hardware," in *SOCC*, 2022.
- [132] M. A. Jibril, H. Al-Sayeh, and K.-U. Sattler, "Accelerating Aggregation Using a Real Processing-in-Memory System," in *ICDE*, 2024.
- [133] J. Chen, J. Gómez-Luna, I. El Hajj, Y. Guo, and O. Mutlu, "SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory," in *PACT*, 2023.
- [134] C. Lim, S. Lee, J. Choi, J. Lee, S. Park, H. Kim, J. Lee, and Y. Kim, "Design and Analysis of a Processing-in-DIMM Join Algorithm: A Case Study with UPMEM DIMMs," in *SIGMOD*, 2023.
- [135] S. Cai, B. Tian, H. Zhang, and M. Gao, "PIM-PAM: Efficient Graph Pattern Matching on Real Processing-in-Memory Hardware," in *SIGMOD*, 2024.
- [136] J. Nider, C. Mustard, A. Zoltan, J. Ramsden, L. Liu, J. Grossbard, M. Dashti, R. Jodin, A. Ghiti, J. Chauzi *et al.*, "A Case Study of Processing-in-Memory in Off-the-Shelf Systems," in *USENIX ATC*, 2021.
- [137] H. Kang, Y. Zhao, G. E. Blelloch, L. Dhulipala, Y. Gu, C. McGuffey, and P. B. Gibbons, "PIM-Tree: A Skew-Resistant Index for Processing-in-Memory," in *VLDB*, 2025.
- [138] H. Kim, Y. Zhao, A. Pavlo, and P. B. Gibbons, "No Cap, This Memory Slaps: Breaking Through the Memory Wall of Transactional Database Systems with Processing-in-Memory," in *VLDB*, 2025.
- [139] W. Kong, S. Zheng, Y. Hua, R. Ma, Y. Wen, G. Wang, C. Zhou, and L. Huang, "PimBeam: Efficient Regular Path Queries Over Graph Database Using Processing-in-Memory," *TPDS*, 2025.
- [140] M. Barkhordar, A. Tabatabaiean, M. Sadrosadati, C. Giannoula, J. G. Luna, I. El Hajj, O. Mutlu, and A. R. Alameldeen, "ALPHA-PIM: Analysis of Linear Algebraic Processing for High-Performance Graph Applications on a Real Processing-In-Memory System," in *IISWC*, 2025.
- [141] S. Yu, H. Kim, K. Jeun, S. Hwang, S. Cho, and E. Lee, "ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration," in *MICRO*, 2025.
- [142] S. U. Noh, J. Hong, C. Lim, S. Park, J. Kim, H. Kim, Y. Kim, and J. Lee, "PIDComm: A Fast and Flexible Collective Communication Framework for Commodity Processing-in-DIMM Devices," in *ISCA*, 2024.
- [143] D. Lee, B. Hyun, T. Kim, and M. Rhu, "PIM-MMU: A Memory Management Unit for Accelerating Data Transfers in Commercial PIM Systems," in *MICRO*, 2024.
- [144] Y. Zhao, M. Gao, F. Liu, Y. Hu, Z. Wang, H. Lin, J. Li, H. Xian, H. Dong, T. Yang *et al.*, "UM-PIM: DRAM-Based PIM with Uniform and Shared Memory Space," in *ISCA*, 2024.
- [145] D. Teguia, J. Chen, S. Bitchebe, O. Balmau, and A. Tchana, "vPIM: Processing-in-Memory Virtualization," in *Middleware*, 2024.
- [146] C. Giannoula, N. Vijaykumar, N. Papadopoulou, V. Karakostas, I. Fernandez, J. Gómez-Luna, L. Orosa, N. Koziris, G. Goumas, and O. Mutlu, "Syncron: Efficient Synchronization Support for Near-Data-Processing Architectures," in *HPCA*, 2021.
- [147] J. Shin, S. An, S. Lee, and S. E. Lee, "PIMCoSim: Hardware/Software Co-Simulator for Exploring Processing-in-Memory Architectures," *Electronics*, 2024.
- [148] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology," in *HPCA*, 2024.
- [149] J. Heo, Y. Shin, S. Choi, S. Yune, J.-H. Kim, H. Sung, Y. Kwon, and J.-Y. Kim, "Primo: A Full-Stack Processing-in-DRAM Emulation Framework for Machine Learning Workloads," in *ICCAD*, 2023.
- [150] J. Kim, D. Kim, S. Kang, B. Hyun, I. Lee, and Y. Park, "PIM-CCA: An Efficient PIM Architecture with Optimized Integration of Configurable Functional Units," in *MICRO*, 2025.
- [151] G. F. Oliveira, J. Gómez-Luna, L. Orosa, S. Ghose, N. Vijaykumar, I. Fernandez, M. Sadrosadati, and O. Mutlu, "DAMOV: A New Methodology and Benchmark Suite for Evaluating Data Movement Bottlenecks," *IEEE Access*, 2021.
- [152] G. Singh, J. Gómez-Luna, G. Mariani, G. F. Oliveira, S. Corda, S. Stuijk, O. Mutlu, and H. Corporaal, "Napel: Near-Memory Computing Application Performance Prediction via Ensemble Learning," in *DAC*, 2019.
- [153] S. Nakandala, K. Saur, G.-I. Yu, K. Karanasos, C. Curino, M. Weimer, and M. Interlandi, "A Tensor Compiler for Unified Machine Learning Prediction Serving," in *OSDI*, 2020.
- [154] A. Sabne, "XLA: Compiling Machine Learning for Peak Performance," in *Google Research Publications*, 2020.
- [155] S. Pouget, M. Lo, L.-N. Pouchet, and J. Cong, "Holistic Optimization Framework for FPGA Accelerators," in *TODAES*, 2025.

A. Artifact Appendix

A.1. Abstract

The Artifact Appendix describes how to reproduce the main results of this paper. It includes the source code of DCC, benchmark scripts, and step-by-step instructions for the key evaluation results. The experiments require a server equipped with a CPU with at least 64 hardware threads, 128GB of main memory, disk space of at least 128GB, and an NVIDIA GPU with an up-to-date NVIDIA driver installed. Note that our artifact needs Anaconda 25.6.1+ and CMake 3.16.3 to be installed in the server. We provide a README.md file that describes the required hardware and software dependencies and provides step-by-step instructions. This artifact is used to support our major claims (See Appendix §A.6), demonstrating DCC’s performance benefits in Figures 7, 8 and 10. We expect the full evaluation pipeline, including the setup and simulations, to take approximately 7-10 days. We also expect that the trace files generated for trace-driven simulations take approximately 90GB disk space.

A.2. Artifact Check-list (meta-information)

- **Program:** *DCC_Artifact*: In this artifact, we evaluate DCC compiler upon HBM-PIM and AttAcc PIM backends and compare HBM-PIM, HBM-PIM + DCC, AttAcc, AttAcc + DCC comparison points against GPU baseline.
- **Compilation:** This artifact **strictly** requires CMake 3.16.3 and GCC/G++ 11.4.
- **Models:** The workloads for kernel performance and inference performance have been configured to evaluate the GPT3-13B and LLAMA2-33B models.
- **Run-time environment:** Linux Ubuntu 20.04 (or newer) with Python 3.8, requiring CUDA 12.3 (or newer).
- **Hardware:** A server equipped with a CPU of at least 64 hardware threads and 128GB of main memory, and an NVIDIA GPU device with a minimum compute capability of 8.0 and at least 8GB GPU memory should be used to validate the results.
- **Execution:** Trace-driven simulations.
- **Metrics:** Execution time normalized as relative performance speedup.
- **Output:** Experimental results are stored as CSV files, and our scripts generate corresponding figures in PDF format. The generated figures are similar to the Figures 7, 8, and 10 of the main paper.
- **Experiments:** Kernel performance and end-to-end inference performance. Detailed steps for the reproduction of evaluation experiments are provided in the README.md file.
- **How much disk space required (approximately)?:** 128 GB.
- **How much time is needed to prepare workflow (approximately)?:** 10 minutes (build the code and download Python packages).
- **How much time is needed to complete experiments (approximately)?:** 7-10 days (run simulations and generate figures).
- **Publicly available?:** Yes.
- **Code licenses:** MIT.

- **Archived:** <https://doi.org/10.5281/zenodo.19442321>

A.3. Description

A.3.1. How to Access. Download the compressed file DCC_Artifact.zip from the Zenodo archive <https://doi.org/10.5281/zenodo.19442321> or our GitHub repository at <https://github.com/SPIN-Research-Group/DCC>.

A.3.2. Hardware Dependencies. The artifact should be tested on a sever with:

- x86-64 CPU with at least 64 hardware threads, 128GB of main memory and 128GB of disk storage.
- NVIDIA GPU device with a minimum compute capability (SM) of 8.0 and at least 8GB GPU memory.

A.3.3. Software Dependencies. The artifact requires the following software for installation:

- Ubuntu 20.04 (or newer)
- Python 3.8
- GNU compilers (gcc/g++) 11.4.0 (**strict** requirement)
- CUDA 12.3 (or newer)
- CMake 3.16.3 (**strict** requirement)
- Anaconda 25.6.1 (or newer)
- Package scikit-learn 1.3.2 (or newer)
- Package XGBoost 2.1.4 (or newer)

A.4. Installation

Download the zip file containing the artifact source code in §A.3.1. We provide detailed instructions in the README.md file under the root of source code directory to build the simulator and evaluate DCC on two PIM backends.

We next summarize the key steps:

1. Install Anaconda 25.6.1 (or newer) following the instructions provided in <https://www.anaconda.com/docs/getting-started/main>. You may need to run the following command after installation to enable conda to create a new virtual environment. Please refer to <https://www.anaconda.com/docs/getting-started/tos-plugin>.
2. Download and install Cmake 3.16.3 (**strict** requirement) following the instructions provided in <https://cmake.org/download/>. Please note that we have not extensively tested all various CMake versions. We **recommend** using and building our artifact with Cmake 3.16.3.
3. Download the source code of DCC_Artifact (See §A.3.1).
4. Setup the runtime environment for all experiments by running the following bash setup script under the root of source code directory:

```
$ conda tos accept
```

```
$ bash setup.sh
```

A.5. Experiment Workflow

The artifact contains three experiments to conduct the evaluation of the Figures 7, 8, and 10 of the main paper. We have a bash script to launch all the simulations of the three evaluation experiments, collect the raw results and save them under the results/ directory. We also include plotting scripts to parse

the raw results and generate the figures under the `figures/` directory. Next, we provide more details.

Launch Experiments & Visualize the Results: We strongly **recommend** using a server with at least 64 hardware threads and at least 128GB of main memory. The following script (i) launches all the experiments required to reproduce the key results of our paper, (ii) stores the raw results under the `results/` directory, and (iii) generates the Figures 7 and 8, 10 under the `figures/` directory:

```
$ bash run_experiments.sh
```

Relaunch Failed Experiments (if any): If there are any failed simulations (the plotting scripts may also fail to visualize the figures), re-run the main bash script:

```
$ bash run_experiments.sh
```

A.6. Evaluation and Expected Result

Major Claims. For each of the three experiments and for the same workload configuration, we expect the reproduced results to be similar to those reported in the paper within $\pm 4\%$ due to randomness in the DCC’s predictor offline training process. Specifically, DCC’s predictor is trained offline by selecting a small set of *random* workload configurations (input token sizes, output token sizes, batch sizes etc.). We will focus on the range of speedup values in each evaluation experiment to verify the reproduction of the key results. We next clarify our major claims:

1. Figure 7: AttAcc + DCC achieves kernel performance speedups for the workload configurations of the ATTN, GEMV and RED kernels in the range of $9.22\times\text{--}13.70\times$, $6.55\times\text{--}10.18\times$, and $0.95\times\text{--}4.99\times$, respectively, over GPU, and in the range of $1.01\times\text{--}1.64\times$, $1.00\times\text{--}1.80\times$, and $1.02\times\text{--}1.73\times$, respectively, over AttAcc.
2. Figure 8: HBM-PIM + DCC achieves kernel performance speedups for the workload configurations of the GEMV, RED, VA, and RELU kernels in the range of $4.63\times\text{--}7.99\times$, $0.80\times\text{--}3.48\times$, $1.29\times\text{--}5.58\times$, and $1.60\times\text{--}6.96\times$, respectively, over GPU, and in the range of $1.18\times\text{--}2.45\times$, $1.35\times\text{--}2.44\times$, $1.35\times\text{--}2.43\times$, and $1.34\times\text{--}2.42\times$, respectively, over HBM-PIM.
3. Figure 10: Attacc_{Base} + DCC achieves end-to-end performance speedups for the workload configurations of the two LLMs in the range of $1.15\times\text{--}1.73\times$ over GPU, and in the range of $1.06\times\text{--}1.41\times$ over Attacc_{Base}. Attacc_{Full} + DCC achieves end-to-end performance speedups in the range of $2.77\times\text{--}8.02\times$ over GPU, and in the range of $1.20\times\text{--}2.85\times$ over Attacc_{Full}.

A.7. Experiment Customization

In kernel performance experiments, users can customize the workload configuration, including batch size, input and output token sizes and number of heads by adding following function call at the end of the `src/test_configs.py` script, where the `name` parameter should be set to "ATTN", "GEMV", "RED", "VA", or "RELU" depending on the target ML kernel:

```
custom_kernel_sizes(name, batch, input, output, nhead)
```

For LLM inference performance experiments, users can customize the workload configuration, including batch size, input and output token sizes, by first adding following function call at the end of the `src/test_configs.py` script:

```
custom_inference_sizes(batch, input, output)
```

and then by adding following command (in one line) at the end of the `run_experiments.sh` script:

```
conda run -n dcc -live-stream python3
run_inference.py -batch=batch -lin=input
-lout=output
```

Note that the plotting scripts do not support custom experiment configurations. To avoid errors, please comment out the last two lines in the `run_experiments.sh` script, and update the corresponding plotting script accordingly to match the custom configurations.